

Formation ExtJS 8

Ali Koudri - ali.koudri@janus-academy.fr



Ali Koudri

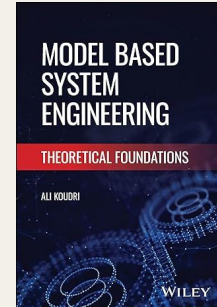
- PhD en informatique
- 25+ ans en ingénierie système et logicielle, modélisation, performance, sécurité
- IRT SystemX - Thales - CEA
- Janus Academy : formation & conseil indépendant
- Approche cross-stack : front, back, ci/cd, observabilité

Publications

Model Based Systems Engineering: Theoretical Foundations

Posture pédagogique

Démarche systémique, mesure avant optimisation, capitalisation transposable



Agenda

- **Présentation général du framework**
- **Module 1 - Socle & système de classes**
- **Module 2 - Composants, conteneurs & layouts**
- **Module 3 - Architecture MVVM**
- **Module 4 - Couche données**
- **Module 5 - Grilles**
- **Module 6 - Arbres, formulaires et charts**
- **Module 7 - SPA & UX**
- **Module 8 - Industrialisation**



EXT JS 8

Présentation du framework

Introduction

Au programme

- **Le framework & ses motivations** Ce qu'est ExtJS, pourquoi, sa philosophie.
- **Concepts fondateurs** La carte de ce que la formation détaillera.
- **Positionnement & écosystème** Cas d'usage, concurrence, licence.
- **Histoire** D'où vient ExtJS, où il en est.

Extrait

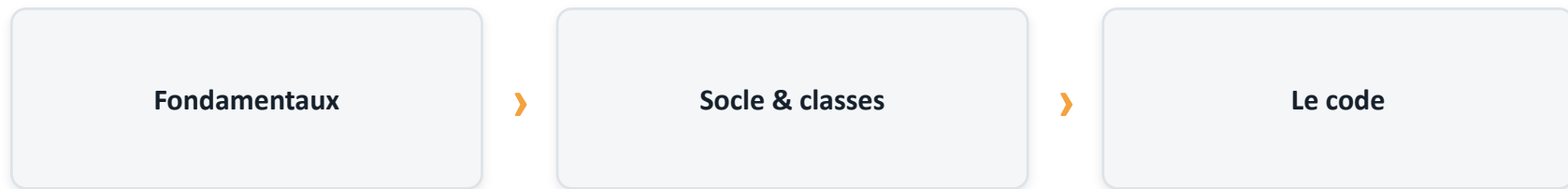
The screenshot displays the ExtJS Kitchen Sink application. The top navigation bar is dark blue with the 'Ext JS Kitchen Sink' logo and a hamburger menu icon. Below this, a blue header bar reads 'Trees - Tree List'. On the left, a sidebar contains a search bar and a list of examples. The 'Tree List' example is highlighted in yellow. The main content area features a 'TreeList' component with a blue header bar containing a gear icon, the title 'TreeList', and buttons for 'Options', 'Nav', and 'Micro'. The tree structure is as follows:

- Home
 - Messages
 - Archive (highlighted)
 - Music
 - Video
- Users
 - Tagged
 - Inactive
 - Groups
 - Settings

To the right of the tree, the text 'Selected: /Home/Archive' is displayed. A vertical 'Details' sidebar is visible on the far right.

Avant d'écrire une ligne

Ce module situe ExtJS avant d'entrer dans le code : savoir ce que l'on adopte, ses forces, ses limites et sa place dans l'écosystème.



Objectif : décider et coder en connaissance de cause, sans dogme ni argument marketing.

À l'issue de ce module

- 1 Comprendre ce qu'est ExtJS et le problème qu'il résout.
- 2 Le situer face à ses alternatives, sans dogme.
- 3 Cartographier ses concepts.
- 4 Connaître son histoire, sa licence et son éditeur.

Le framework & ses motivations

Qu'est-ce qu'ExtJS ?

- **Framework d'application web** Pour construire des applications monopage (SPA) riches.
- **Orienté data-intensive** Grilles, formulaires, arbres, dashboards, multi-plateformes.
- **Batteries incluses** Composants, couche données et outillage d'un seul fournisseur.
- **Deux usages** Simple bibliothèque de composants, ou framework SPA complet.

Le problème qu'il résout

- **Coût du data-intensive** Grilles et formulaires complexes sont chers à bâtir à la main.
- **Assemblage hétérogène** Réunir des bibliothèques disparates fragilise la maintenance.
- **Cible entreprise** Back-office et outils métier au cycle de vie long.
- **Priorité** Cohérence et durée plutôt que légèreté et minimalisme.

Philosophie

- **Batteries incluses** Le nécessaire dans un socle unique et cohérent.
- **Convention forte** Nommage = chemin ; structure imposée par Sencha Cmd.
- **Système de classes** Héritage, mixins et config system par-dessus JavaScript.
- **Orienté données** Models et stores irriguent les composants.

Avantages

- **Composants riches** Grilles, arbres, charts, formulaires prêts à l'emploi.
- **Cohérence** Un modèle unique : classes, composants, données, MVVM.
- **Outillage intégré** Build, tests, thèmes, inspecteur d'un même fournisseur.
- **Pérennité contractuelle** Support commercial et feuille de route soutenue.

Limites & coûts

- **Poids** Bundle volumineux : inadapté à une vitrine légère.
- **Licence** Coût par développeur, désormais par abonnement.
- **Courbe d'apprentissage** Paradigme propre : config system, layouts, build.
- **Marché & recrutement** Vivier plus étroit que React/Angular ; dépendance à un éditeur unique.

Concepts fondateurs

La carte des concepts

- **Système de classes** Ext.define, config, mixins → Module 1.
- **Composants & layouts** Conteneurs et mise en page → Module 2.
- **Architecture MVVM** Vue, ViewController, ViewModel → Module 3.
- **Couche données** Models, stores, proxies → Module 4.
- **Grilles, arbres, charts** Composants orientés données → Modules 5-6.
- **Production** Build, tests, thèmes, i18n → Modules 7-8.

Un avant-goût

Le cœur d'ExtJS : déclarer des classes. Ce qui suit sera détaillé au Module 1 ; ici, seulement en reconnaître la forme.

```
Ext.define('App.model.Equipement', {  
  extend: 'Ext.data.Model',  
  fields: [  
    { name: 'nom', type: 'string' },  
    { name: 'etat', type: 'string' }  
  ]  
});
```

Composants & layouts

- **Tout est composant** Organisé en conteneurs imbriqués.
- **Le layout place les enfants** Il dimensionne ; le développeur choisit la stratégie.
- **Bibliothèque riche** Panels, grilles, arbres, formulaires, fenêtres, charts.
- **Événements unifiés** Un même modèle relie les composants entre eux.

Données & MVVM

- **Models** La forme et les règles d'un enregistrement.
- **Stores** Des collections alimentées par des proxies (serveur).
- **MVVM** Vue déclarative, ViewController (comportement), ViewModel (état).
- **Databinding** L'interface reflète l'état sans code de synchronisation.

L'outillage

- **Sencha Cmd** Scaffold, build incrémental, packages.
- **Sencha Inspector** Inspection de l'arbre de composants et des bindings.
- **Sencha Test** Tests unitaires et end-to-end.
- **Thèmes & IDE** Fashion pour le thème, plugin JetBrains pour l'édition.

Positionnement & écosystème

Cas d'usage

- **Back-office & outils métier** Applications internes de gestion.
- **Supervision & dashboards** Consoles data-intensive, l'esprit du fil rouge de cette formation.
- **Formulaires & grilles complexes** Saisie, validation, édition de masse.
- **Secteurs au cycle long** Finance, industrie, santé ; maintenance sur la durée.

Face à la concurrence

- **React / Angular / Vue** Écosystèmes ouverts : composants à assembler, souvent via des tiers.
- **Grilles & composants entreprise** AG Grid, Kendo UI, DevExtreme, Vaadin, Infragistics.
- **La différence ExtJS** Un framework « tout compris » d'un fournisseur commercial unique.
- **Tendance** React/Angular dominant le neuf ; ExtJS tient le back-office et le legacy.

ExtJS, ou autre chose ?

ExtJS pertinent

- App interne data-intensive (grilles, formulaires).
- Cycle de vie long, socle unique et supporté.
- Budget de licence assumé.

Préférer autre chose

- Site vitrine ; empreinte/perf critiques.
- Équipe déjà investie React / Angular.
- Besoin d'un large vivier open source.

Le bon outil dépend du contexte, pas d'une préférence : ce module aide à la décision.

Licence & éditeur

- **Éditeur** Sencha, filiale d'IDERA depuis 2017.
- **Community Edition** Gratuite, restreinte par le chiffre d'affaires.
- **Éditions payantes** Pro, Enterprise... en abonnement annuel par développeur.
- **Depuis avril 2026** Plus de nouvelles licences perpétuelles ; runtime sans redevance.

Histoire

D'où vient ExtJS

- **2007** Jack Slocum publie Ext JS, né de YUI-Ext.
- **2010** Naissance de Sencha (fusion Ext JS, jQTouch, Raphaël).
- **2011** Ext JS 4 : nouveau système de classes, chargement dynamique, MVC.
- **2015** Ext JS 6 : fusion des toolkits classic & modern.
- **2017** Sencha acquis par IDERA.
- **2018** Community Edition gratuite.
- **2026** Ext JS 8 ; fin des nouvelles licences perpétuelles.

Les apports, version par version (1/2)

- **Ext JS 1 - 2007**

Première version (Jack Slocum, issu de YUI-Ext) : grille et widgets riches pour le web.

- **Ext JS 2 - 2007**

Refonte de l'API des composants ; utilisable en autonome ou via adaptateurs (YUI, jQuery, Prototype).

- **Ext JS 3 - 2009**

Arrivée des graphiques et de la list view ; compatibilité ascendante avec la 2.0.

- **Ext JS 4 - 2011**

Refonte majeure : nouveau système de classes, chargement dynamique, architecture MVC, charts HTML5.

Les apports, version par version (2/2)

- **Ext JS 5 - 2014**

Passage à MVVM et two-way data binding ; layouts responsive ; widgets en cellule de grille.

- **Ext JS 6 - 2015**

Fusion d'Ext JS et Sencha Touch : toolkits classic & modern (desktop + mobile, socle unique).

- **Ext JS 7 - 2019**

Modernisation du toolkit modern (grille, éditeur Froala) ; convergence classic/modern (lignée 7.x jusqu'en 2025).

- **Ext JS 8 - 2026**

Lecteur/générateur de QR code, pavé de signature ; buffering horizontal de colonnes.

Ce qui est posé, ce qui vient

Posé

- Ce qu'est ExtJS, ses motivations.
- Carte des concepts.
- Positionnement & licence.
- Histoire du framework.

À venir

- On entre dans le code.
- Le système de classes.
- Le fil rouge : VIGIE, une console de supervision.
- Déploiement industriel



VIGIE

Socle & système de classes

ExtJS 8 · MODULE 1

Au programme

- **Bloc 1.1 - Pourquoi un système de classes ?** Ce qu'ExtJS ajoute à JavaScript moderne.
- **Bloc 1.2 - Environnement & outillage** Exécuter, déboguer, naviguer la documentation.
- **Bloc 1.3 - Système de classes** Ext.define, config system, héritage, mixins, chargement.
- **Lab M1 - Bootstrap de VIGIE** Scaffold de l'application, première classe métier.

D'où l'on part, où l'on va

Prérequis JO acquis : JavaScript moderne (ES6+, prototypes, closures, async/Promises, modules).



Objectif de cette première partie : disposer du modèle mental et de l'outillage pour bâtir VIGIE sur des fondations saines.

À l'issue de cette première partie

- 1 Maîtriser l'environnement : exécution, débogage, documentation API.
- 2 Définir et instancier des classes avec Ext.define et le config system.
- 3 Structurer la réutilisation par héritage et par mixins, à bon escient.
- 4 Scaffold le squelette de VIGIE avec Sencha Cmd.

● BLOC 1.1

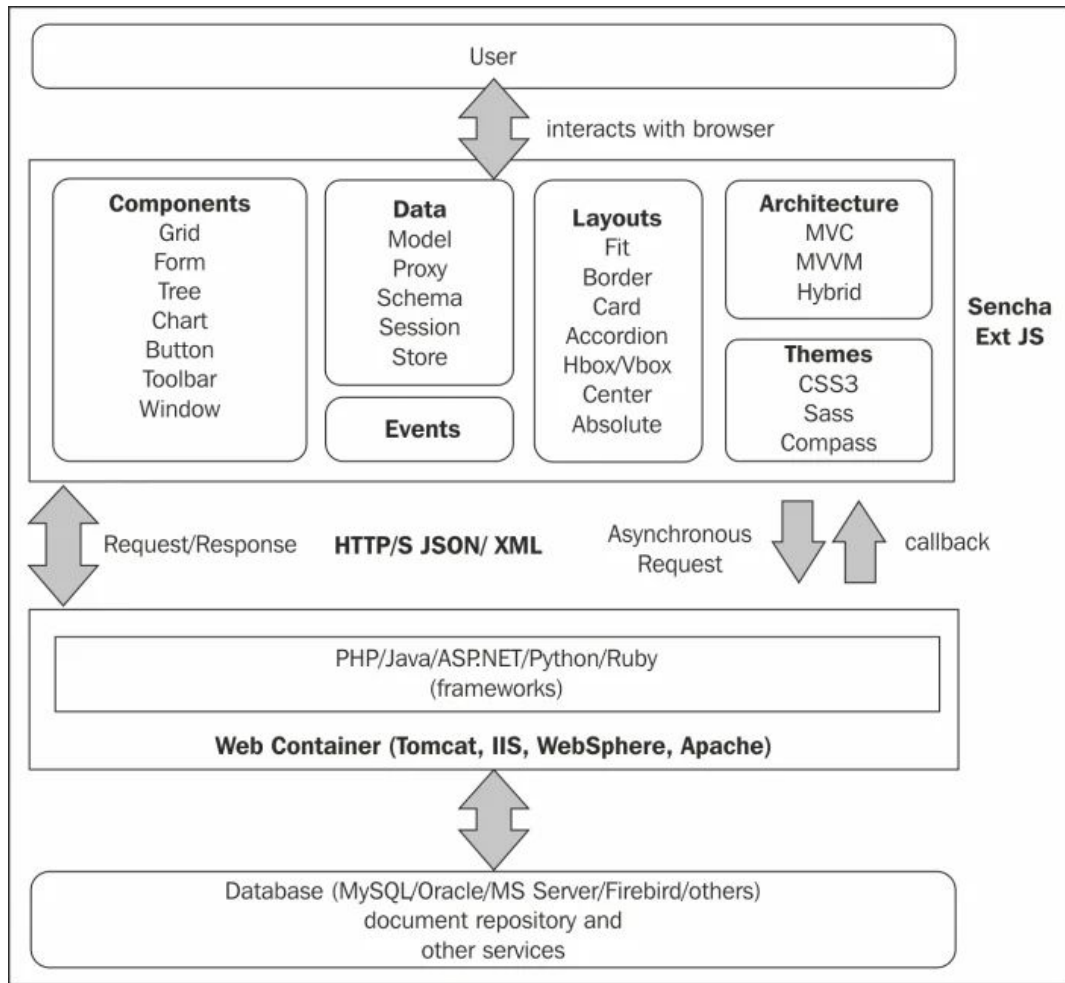
Pourquoi un système de classes ?

ExtJS n'est pas qu'une bibliothèque d'UI

JavaScript ES6 fournit déjà classes, modules et async. ExtJS apporte une couche d'application cohérente par-dessus :

- **Config system** Propriétés déclaratives avec getters/setters, apply/update générés.
- **Mixins & overrides** Composition transverse au-delà de l'héritage simple.
- **Chargement par dépendances** requires + Ext.Loader : résolution avant instantiation.
- **Composants & layout** Un modèle de rendu et de mise en page industrialisé.

ExtJS : Architecture



Quand ExtJS, quand rester léger

ExtJS se justifie

- Application data-intensive (grilles, arbres, formulaires liés).
- Back-office d'entreprise au cycle de vie long.
- Besoin d'un socle UI homogène et outillé.

Préférer plus léger

- Page vitrine ou widget isolé.
- Empreinte / temps de chargement critiques.
- Équipe déjà investie dans un autre écosystème.

***Règle** : choisir sur des contraintes réelles, pas par habitude ni par dogme.*

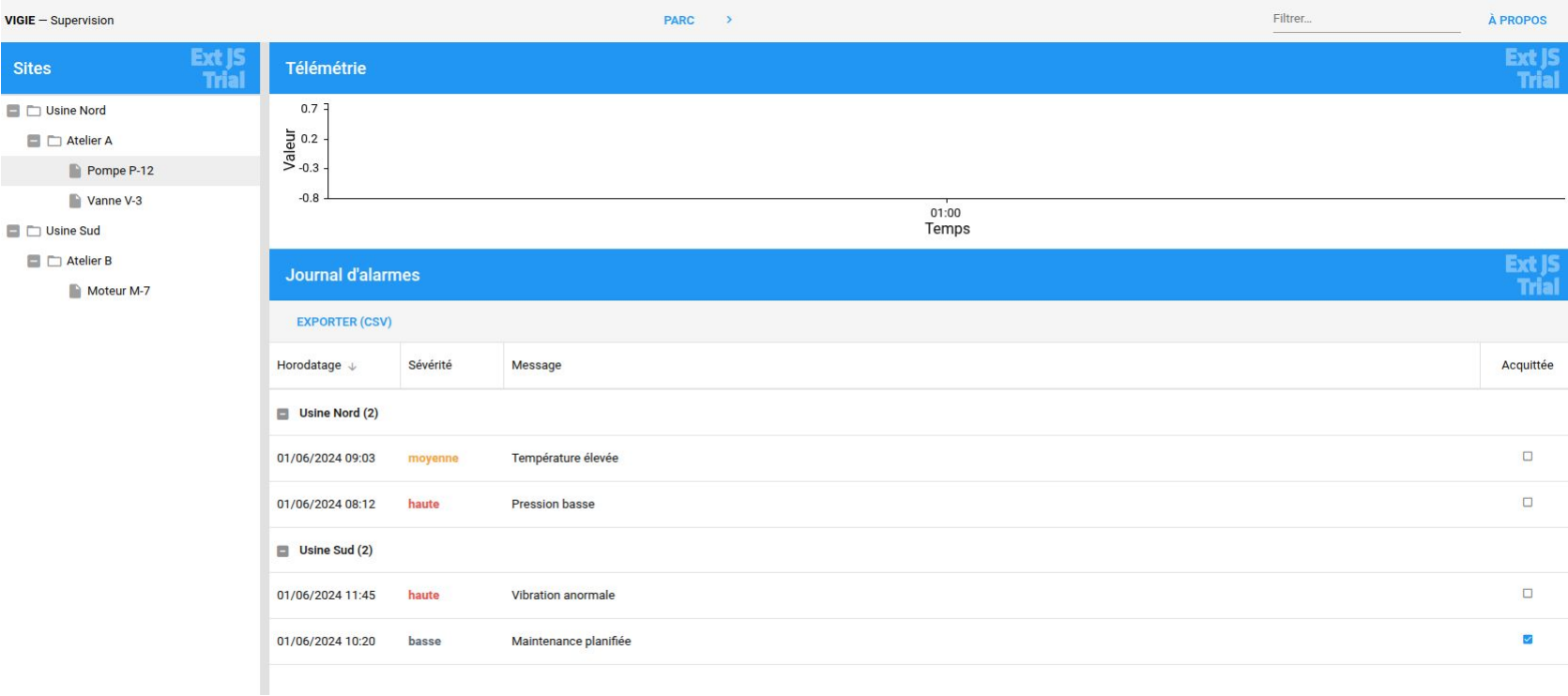
VIGIE - la cible

Console de supervision d'un parc d'équipements. Le même artefact grandit du M1 au M8.

- **Hiérarchie** Site → Zone → Équipement (arbre + fil d'Ariane).
- **Surveillance** Grille d'équipements et journal d'alarmes.
- **Télémétrie** Mesures visualisées en graphiques sur un dashboard.
- **Configuration** Formulaires liés, validations, maître-détail.



VIGIE - la cible



● BLOC 1.2

Environnement & outillage

Structure d'un projet ExtJS

- **app/** Le code : model, store, view, controller (convention de nommage = chemin).
- **app.json** Manifeste : dépendances, toolkit, thème, requires globaux.
- **ext/** Le framework lui-même, géré par Sencha Cmd.
- **classic/ · modern/** Surcharges spécifiques au toolkit ciblé.

***Convention** : la classe `VIGIE.view.Dashboard` se trouve dans `app/view/Dashboard.js`.*

BLOC 1.2

Structure du projet VIGIE

```

  app
  ├── controller
  │   └── JS Navigation.js
  ├── domain
  │   └── JS Equipement.js
  ├── model
  │   ├── JS Alarme.js
  │   ├── JS Equipement.js
  │   └── JS Mesure.js
  ├── overrides
  ├── sass
  │   ├── src.scss
  │   └── var.scss
  ├── store
  │   ├── JS Alarmes.js
  │   ├── JS Arborescence.js
  │   ├── JS Equipements.js
  │   └── JS Mesures.js
  ├── util
  │   └── JS SeuilTelemetrie.js
  ├── view
  │   ├── alarmes
  │   │   └── JS Journal.js
  │   └── main
  │       ├── JS Main.js
  │       ├── JS MainController.js
  │       └── JS MainModel.js
  ├── JS Application.js
  ├── JS Libelles.js
  └── JS Locale.js

```

Exécuter & déboguer

- **sencha app watch** Build incrémental + rechargement, serveur de dev local.
- **Sencha Inspector** Inspection de l'arbre de composants, des stores et des bindings.
- **Plugin JetBrains** Complétion et navigation dans les classes Ext sous WebStorm/IDEA.

```
$ sencha generate app -ext VIGIE ./vigie
$ cd vigie
$ sencha app watch
# → http://localhost:1841
```

Lire la documentation API

- **Configs vs Properties vs Methods** Trois onglets distincts : ne pas confondre une config et une méthode.
- **Events** Le contrat d'événements d'un composant - central pour le MVVM (M2).
- **Héritage affiché** La chaîne extend + les mixins appliqués sont explicités en tête de page.
- **Sélecteur de version** Vérifier 7.x vs 8 : certaines API diffèrent.

<https://docs.sencha.com/extjs/8.0.0/modern/Ext.html>

Lire la documentation API

The screenshot shows the Ext JS 8.0.0 API documentation page. The header is dark blue with the text "Ext JS 8.0.0 - Modern Toolkit" and a search bar. The left sidebar is dark blue with "Guides" and "API" tabs, and a list of Ext namespaces like "app", "calendar", "carousel", etc. The main content area is white with a "Summary" section. It describes the Ext namespace, lists user interface components, and provides a code example for `Ext.application()`. A right sidebar shows the "NPM Package" information for `@sencha/ext-core`.

Ext JS 8.0.0 - Modern Toolkit

History :

Guides API

modern classic

Ext

- app
- calendar
- carousel
- chart
- d3
- data
- dataview
- device
- direct
- dom
- drag
- draw
- enums
- event
- exporter
- field

Ext SINGLETON

configs 3 properties 65 methods 137

Public Protected Private Inherited Read Only

Filter members...

Summary

The Ext namespace (global object) encapsulates all classes, singletons, and utility methods provided by Sencha's libraries.

Most user interface Components are at a lower level of nesting in the namespace, but many common utility functions are provided as direct properties of the Ext namespace.

Also many frequently used methods from other classes are provided as shortcuts within the Ext namespace. For example `Ext.getCmp` aliases `Ext.ComponentManager.get`.

Many applications are initiated with `Ext.application` which is called once the DOM is ready. This ensures all scripts have been loaded, preventing dependency issues. For example:

```
Ext.application({
    name: 'MyApp',

    launch: function () {
        Ext.Msg.alert(this.getName(), 'Ready to go!');
    }
});
```

[Sencha Cmd](#) is a free tool for helping you generate and build Ext JS (and Sencha Touch) applications. See [Ext.app.Application](#) for more information about creating an app.

A lower-level technique that does not use the `Ext.app.Application` architecture is `Ext.onReady`.

You can also discuss concepts and Issues with others on the [Sencha Forums](#).

NPM Package
@sencha/ext-core

Hierarchy
└─ Ext

● BLOC 1.3

Système de classes

Ext.define & Ext.create

Ext.define déclare une classe (nom = chemin) ; Ext.create l'instancie en passant par le config system.

```
Ext.define('VIGIE.model.Equipement', {  
    extend: 'Ext.data.Model',  
    fields: [  
        { name: 'nom', type: 'string' },  
        { name: 'etat', type: 'string' }  
    ]  
});
```

Instancier & accéder aux données

```
var eq = Ext.create('VIGIE.model.Equipement', {  
    nom: 'Pompe P-12',  
    etat: 'OK'  
});  
  
eq.get('nom');    // 'Pompe P-12'  
eq.set('etat', 'DEFAULT');
```

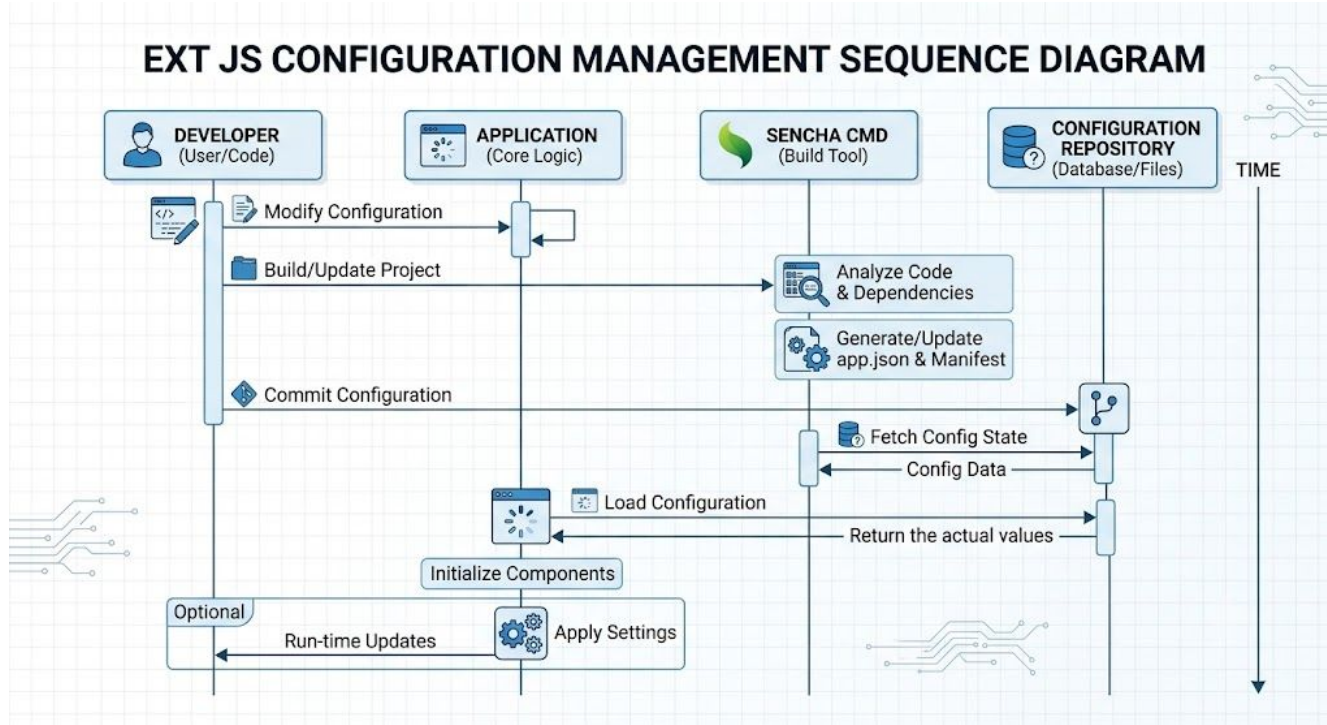
Les champs ne s'écrivent pas directement : get/set passent par la couche données (validations, dirty tracking).

Le config system

Déclarer une propriété dans config génère automatiquement getter et setter, et permet d'intercepter :

- **applyXxx(v)** Normalise / valide la valeur avant affectation (peut la transformer).
- **updateXxx(v, old)** Réagit après changement (effet de bord, notification).
- **initConfig(cfg)** À appeler dans le constructeur pour amorcer les configs.

Le config system : Fonctionnement



Config system - apply / update

```
Ext.define('VIGIE.Capteur', {  
  config: { seuil: 80 },  
  
  applySeuil: function (v) {  
    return Math.max(0, Math.min(100, v));  
  },  
  updateSeuil: function (v, old) {  
    console.log('seuil', old, '->', v);  
  }  
});
```

Héritage : extend & callParent

extend établit la chaîne parent ; **callParent** appelle l'implémentation héritée sans la nommer en dur.

```
Ext.define('VIGIE.Pompe', {  
    extend: 'VIGIE.Equipement',  
  
    statut: function () {  
        return 'pompe:' + this.callParent();  
    }  
});
```

Mixins & membres statiques

- **mixins** Composer des comportements transverses sans imposer une hiérarchie.
- **statics** Membres de classe (factory, constantes) partagés par toutes les instances.
- **Conflits** Le dernier mixin l'emporte ; arbitrage explicite si collision de méthodes.

Un mixin répond à « ce comportement est partagé par des classes sans ancêtre commun » - typiquement transversal.

Un mixin réutilisable

```
Ext.define('VIGIE.mixin.Horodatable', {  
    horodater: function () {  
        this.maj = new Date();  
        return this.maj;  
    }  
});  
  
Ext.define('VIGIE.Alarme', {  
    mixins: ['VIGIE.mixin.Horodatable']  
});
```

Chargement par dépendances

requires déclare les classes nécessaires ; **Ext.Loader** les résout avant l'instanciation. En production, le build les regroupe.

```
Ext.define('VIGIE.view.Tableau', {  
    extend: 'Ext.grid.Panel',  
    requires: [  
        'VIGIE.store.Equipements'  
    ] // résolu avant le rendu  
});
```

Héritage ou mixin ?

Héritage (extend)

- Relation « est-un » naturelle.
- Spécialisation d'un composant Ext existant.
- Une seule lignée, hiérarchie claire.

Mixin

- Comportement « sait-faire » transverse.
- Partagé par des classes sans ancêtre commun.
- Composition multiple assumée.

Par défaut, préférer la composition (mixin) à une hiérarchie profonde et rigide.

● LAB M1

Bootstrap de VIGIE

Bootstrap de VIGIE

SOCLE - POUR TOUS

- Générer l'app via Sencha Cmd et la faire démarrer.
- Définir la classe VIGIE.model.Equipement (config system).
- L'instancier, lire/écrire un champ, vérifier dans l'Inspector.

EXTENSION - POUR ALLER PLUS LOIN

- Écrire un mixin Horodatable réutilisable.
- Appliquer un override à une classe Ext existante.
- Démontrer l'effet des deux à l'exécution.

État attendu en fin de lab

- L'application VIGIE scaffoldée démarre via sencha app watch.
- Une classe métier est définie avec le config system et instanciée.
- L'environnement de débogage (Inspector) est opérationnel.

***Resynchro** : un dépôt VIGIE de référence (état fin M1) est fourni. Repartir de cet état si besoin.*

Ce qui est acquis, ce qui vient

Acquis - M1

- Outillage et débogage.
- Ext.define / Ext.create.
- Config system.
- Héritage & mixins.

À venir - M2

- Composants & conteneurs.
- Le système de layout (point dur).
- Panels, toolbars, windows.
- Shell applicatif de VIGIE.

● FIN DU Module 1

Place au Lab M1

Prochaine étape : Composants, conteneurs & layouts (Module 2).



VIGIE

Composants, conteneurs & layouts

ExtJS 8 · Module 2

Au programme

- **Bloc 2.1 - Composants & conteneurs** Cycle de vie, conteneurs, référencement.
- **Bloc 2.2 - Le système de layout** Le point dur : qui dimensionne quoi.
- **Bloc 2.3 - Panels, toolbars, windows & événements** Les briques d'interface et leurs interactions.
- **Lab M2 - Shell de VIGIE** Viewport, régions, barre d'outils, fenêtre modale.

Du modèle à l'interface

Acquis du Module 1 : système de classes, config system, héritage et mixins. Mais VIGIE n'a encore aucune interface.



Objectif : assembler des composants dans des conteneurs et maîtriser leur disposition - le socle visuel de VIGIE.

À l'issue de cette partie

- 1 Distinguer composant et conteneur, et situer le cycle de vie.
- 2 Référencer un composant (xtype, itemId) et comprendre l'instanciation paresseuse.
- 3 Maîtriser les layouts essentiels : fit, vbox/hbox, border, card.
- 4 Bâtir le shell de VIGIE et diagnostiquer un problème de dimensionnement.

● BLOC 2.1

Composants & conteneurs

Composant ou conteneur ?

Ext.Component

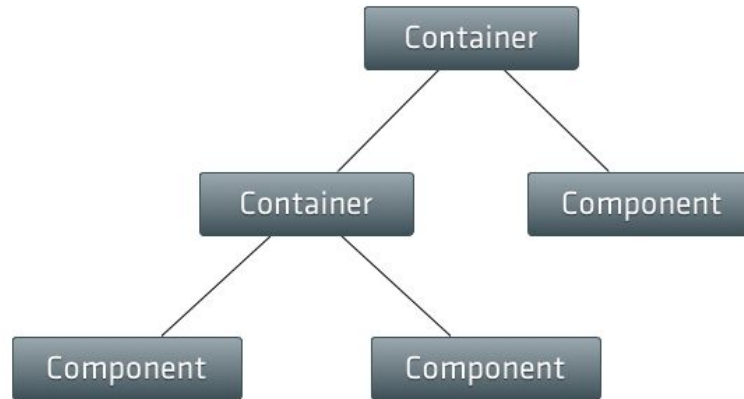
- Une brique d'interface élémentaire.
- Se rend dans le DOM, gère son cycle de vie.
- Ex. : bouton, champ, étiquette.

Ext.container.Container

- Un composant qui contient d'autres composants.
- Délègue leur disposition à un layout.
- Ex. : panel, toolbar, window, viewport.

Tout conteneur est un composant ; l'inverse est faux. La disposition n'existe qu'au niveau conteneur.

Composant vs Conteneur : Illustration

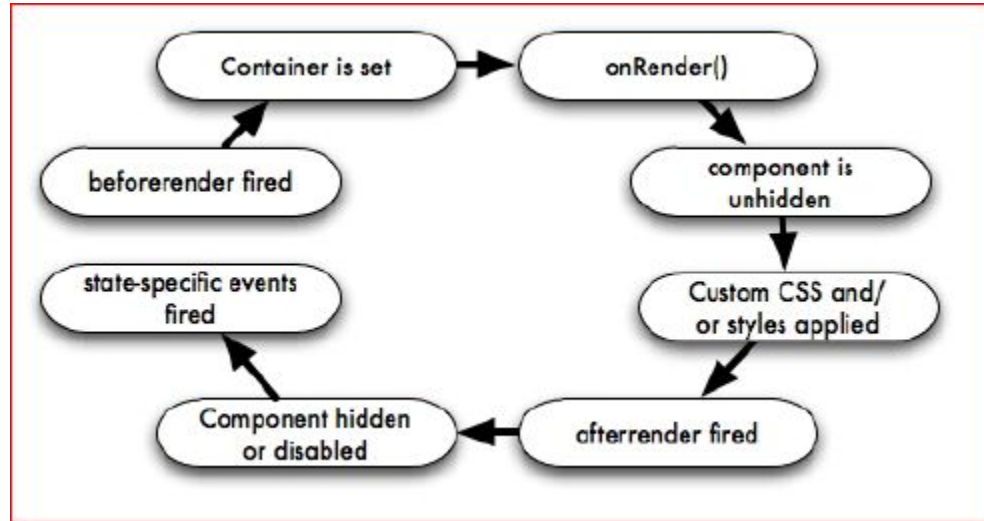


Le cycle de vie d'un composant

- **Construction - `initComponent`** Les configs sont prêtes ; le DOM n'existe pas encore.
- **Rendu - `render` / `afterrender`** Le DOM est disponible ; manipulation et mesures possibles.
- **Destruction - `destroy`** Libérer listeners, stores et références pour éviter les fuites.

Règle : ne jamais toucher au DOM d'un composant avant `afterrender`.

Le cycle de vie d'un composant



Hooks du cycle de vie

```
Ext.define('VIGIE.view.Bandeau', {  
    extend: 'Ext.Component',  
  
    initComponents: function () {  
        this.callParent(); // configs prêtes  
    },  
    listeners: {  
        afterrender: function () { /* DOM ok */ }  
    }  
});
```

Référencer & instancier

Les enfants se déclarent en items via leur xtype (instanciation paresseuse, au rendu). On les retrouve par itemId.

```
items: [  
  { xtype: 'button',    text: 'Rafraîchir', itemId: 'btnMaj' },  
  { xtype: 'tbfill' },  
  { xtype: 'textfield', emptyText: 'Filtrer...' }  
]  
  
panel.down('#btnMaj'); // récupération par itemId
```

Le référencement déclaratif par reference/lookup arrive avec le ViewController (Module 3).

● BLOC 2.2

Le système de layout

Le conteneur ne dimensionne pas seul

Chaque conteneur délègue le placement et la taille de ses enfants à un layout. Le choix du layout EST la décision de mise en page.

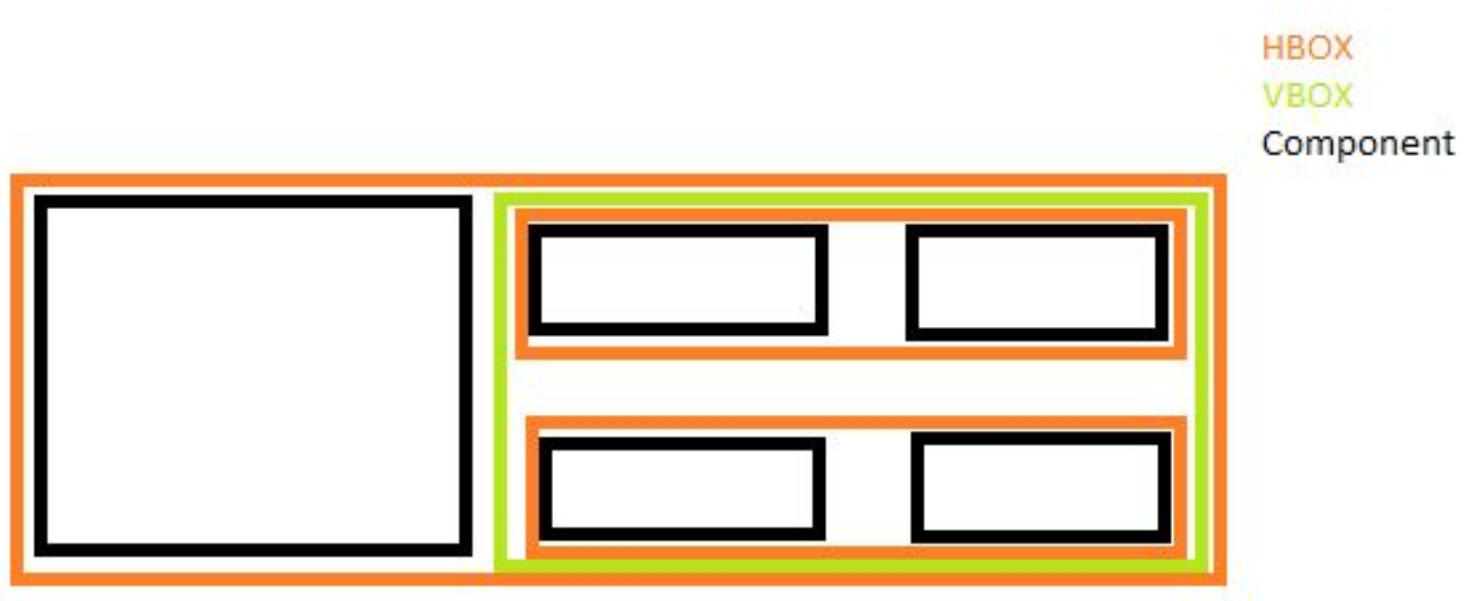
- **fit** Un unique enfant occupe tout l'espace du conteneur.
- **card** Un enfant visible à la fois (assistants, onglets) ; pile commutable.
- **vbox / hbox** Empilement vertical / horizontal avec répartition par flex.
- **border** Régions ancrées : north, south, east, west, center.

vbox / hbox : flex & align

- **flex** Répartit l'espace restant proportionnellement entre les enfants.
- **align: 'stretch'** Étire les enfants sur l'axe transverse (largeur en vbox).
- **Dimensions mixtes** Tailles fixes et flex peuvent coexister sur le même axe.

```
layout: { type: 'vbox', align: 'stretch' },
items: [
  { xtype: 'panel', title: 'Filtres', height: 80 },
  { xtype: 'panel', title: 'Contenu', flex: 1 }
]
```

vbox / hbox : Exemple



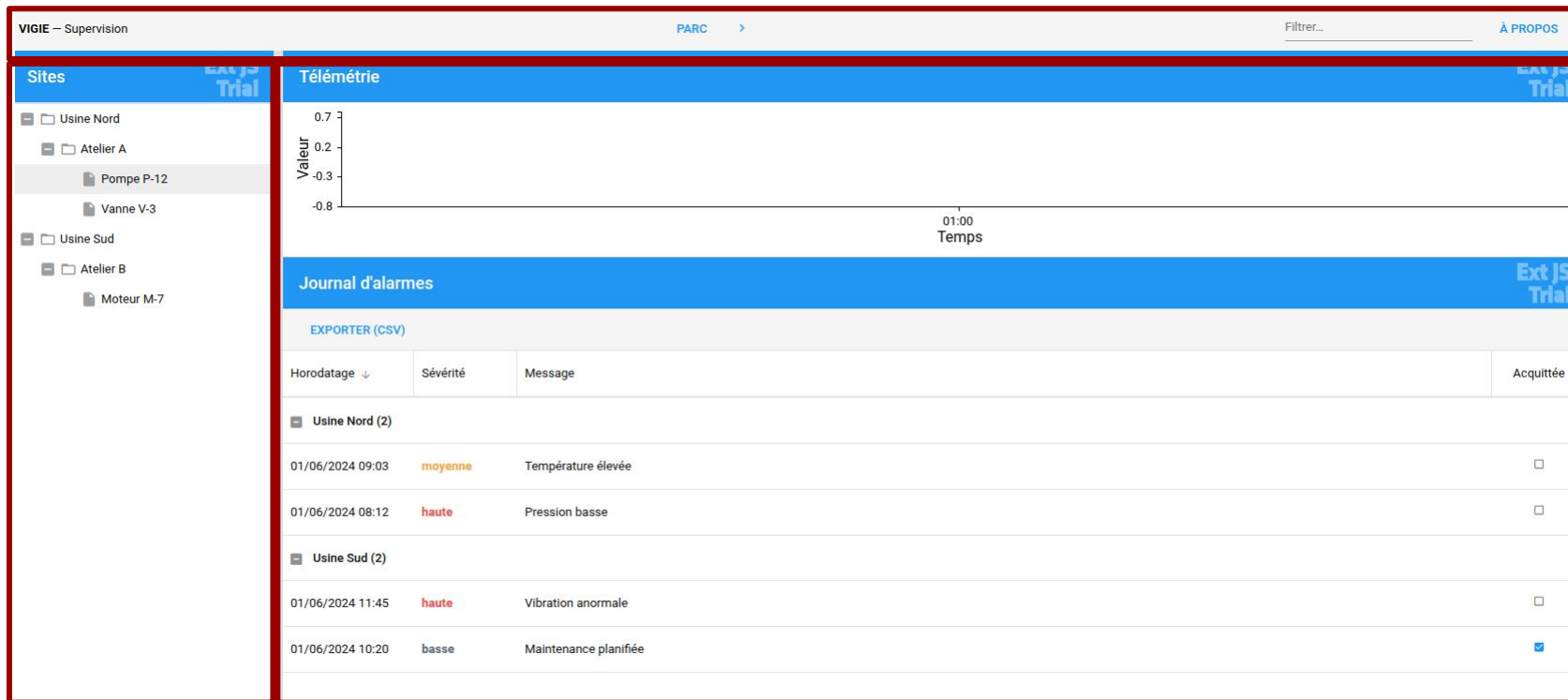
border : l'ossature de VIGIE

Le layout border ancre des régions cardinales. center est obligatoire ; les autres sont redimensionnables et repliables.

```
layout: 'border',
items: [
  { region: 'north', xtype: 'toolbar', height: 48 },
  { region: 'west', title: 'Sites', width: 240,
    collapsible: true, split: true },
  { region: 'center', xtype: 'panel', layout: 'fit' }
]
```

BLOC 2.2

border : l'ossature de VIGIE



Pièges de dimensionnement

- **Hauteur indéterminée + flex** Un parent sans hauteur connue → l'enfant en flex ne s'affiche pas.
- **shrink-wrap surprise** Sans contrainte, un conteneur se dimensionne sur son contenu, pas l'inverse.
- **Double dimensionnement** Mêler taille Ext et CSS sur le même axe produit des conflits silencieux.
- **Layout thrashing** Trop d'invalidations successives → recalculs coûteux ; regrouper les changements.

Layout ExtJS ou CSS ?

Layout ExtJS

- Régions d'application, panneaux redimensionnables.
- Splitters, repli, dimensionnement entre composants.
- Recalcul au redimensionnement de la fenêtre.

Rester en CSS

- Micro-mise en page interne d'un composant.
- Flux typographique, espacements fins.
- Décor purement visuel, sans logique de taille.

Choisir l'outil au bon niveau : le layout pour la structure, le CSS pour le détail. Pas les deux sur le même axe.

● BLOC 2.3

Panels, toolbars, windows & événements

Panels, toolbars & buttons

- **Panel** Le conteneur de référence : titre, outils, docked items, repli.
- **Toolbar** Barre d'actions ; tbfill et tbseparator structurent l'espace.
- **Button** Action simple, menu, groupe à bascule (toggle/segmented).
- **Docked items** Ancrer une barre en haut/bas d'un panel via dockedItems.

Fenêtres modales

Une window est un panel flottant. `modal: true` bloque l'arrière-plan ; `layout: 'fit'` fait épouser le contenu.

```
var win = Ext.create('Ext.window.Window', {  
    title: 'À propos de VIGIE', modal: true,  
    width: 360, height: 200, layout: 'fit',  
    items: [{ xtype: 'component', html: '...' }]  
});  
win.show();
```

Le modèle d'événements

- **listeners** Réagir aux événements d'un composant, en config ou via `on()`.
- **Événements personnalisés** `fireEvent` publie un signal métier que d'autres écoutent.
- **Découplage** L'émetteur ne connaît pas ses auditeurs - clé pour le MVVM (Module 3).

Préférer un événement métier (« équipementchoisi ») à un couplage direct entre composants.

Publier & écouter un événement

```
Ext.define('VIGIE.view.Selecteur', {  
    extend: 'Ext.panel.Panel',  
    selectionner: function (eq) {  
        this.fireEvent('equipementchoisi', this, eq);  
    }  
});  
  
selecteur.on('equipementchoisi', function (src, eq) {  
    console.log('choisi', eq.getNom());  
});
```

● LAB M2

Shell de VIGIE

Shell de VIGIE

SOCLE - POUR TOUS

- Viewport en layout border (nord : barre d'outils).
- Ouest : panneau « Sites » repliable + splitter.
- Centre : zone de contenu en layout fit.
- Fenêtre modale « À propos ».

EXTENSION - POUR ALLER PLUS LOIN

- Layout imbriqué hbox dans vbox (flex + tailles fixes).
- Diagnostiquer puis corriger un enfant invisible.
- Émettre un événement métier depuis la barre d'outils.

État attendu en fin de lab

- Le shell de VIGIE s'affiche : régions border, barre d'outils, zone centrale.
- Le panneau ouest se replie et se redimensionne (splitter).
- La fenêtre modale s'ouvre et bloque l'arrière-plan.

***Resynchro** : dépôt VIGIE de référence (état fin M2) fourni. Repartir de cet état si le layout résiste.*

Ce qui est acquis, ce qui vient

Acquis - M2

- Composants & conteneurs.
- Cycle de vie.
- Layouts (fit, box, border).
- Shell de VIGIE.

À venir - M3

- V / VC / VM.
- ViewController & ViewModel.
- Databinding (quand binder).
- Refactor MVVM de VIGIE.

● FIN DU Module 2

Place au Lab M2

Prochaine étape : Architecture MVVM (Module 3).



VIGIE

Architecture MVVM

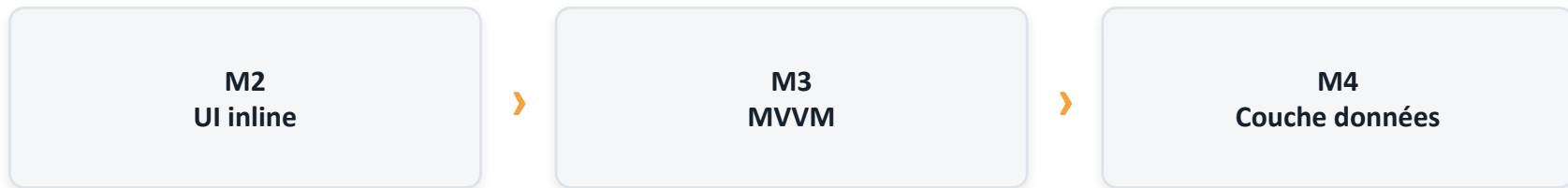
ExtJS 8 · Module 3

Au programme

- **Bloc 3.1 - V / VC / VM** Répartir les responsabilités.
- **Bloc 3.2 - ViewController** Référencement, listeners, événements métier.
- **Bloc 3.3 - ViewModel & databinding** Le point dur : quand binder, quand ne pas.
- **Lab M3 - Refactor MVVM** Extraction VC/VM et maître-détail de VIGIE.

La logique ne peut pas rester dans la vue

Le shell du Module 2 fonctionne, mais ses handlers et son événement vivent en fonctions inline dans la vue : difficile à tester, à réutiliser, à faire évoluer.



Objectif : séparer la structure (vue), le comportement (ViewController) et l'état (ViewModel).

À l'issue de cette partie

1 Répartir les responsabilités entre vue, ViewController et ViewModel.

2 Câbler un ViewController : reference/lookup, listeners, événements.

3 Exposer un état dans un ViewModel et le lier par databinding.

4 Refactorer VIGIE en MVVM et établir un maître-détail.

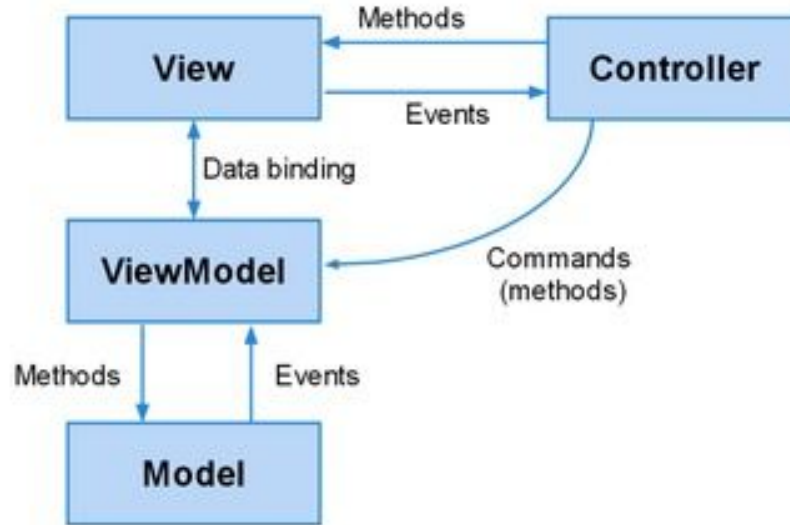
● BLOC 3.1

V / VC / VM

Trois rôles, une vue

- **View** Structure déclarative des composants. Aucune logique métier.
- **ViewController** Comportement local : handlers, orchestration, réactions aux événements.
- **ViewModel** État observable, données dérivées (formulas), stores. La source de vérité du binding.
- **Portée** Un VC et un VM sont attachés à une vue et à son sous-arbre.

Présentation



Ce qui va où

Dans la vue

- Arborescence et configuration des composants.
- Layout, bind, reference.
- listeners pointant vers le ViewController.

Hors de la vue

- Logique applicative → ViewController.
- État & données dérivées → ViewModel.
- Accès serveur → stores / proxies (Module 4).

Anti-pattern n°1 : une vue qui calcule, filtre ou appelle le serveur elle-même.

● BLOC 3.2

ViewController

Le ViewController

- **Attachement** controller: 'main' côté vue ↔ alias: 'controller.main' côté VC.
- **Comportement** Héberge les handlers (référéncés par chaîne) et l'orchestration.
- **Accès** lookup('ref') pour les composants, getViewModel() pour l'état.
- **Cycle de vie** Créé et détruit avec la vue : pas de fuite de listeners.

Référencer sans sélecteur fragile

- **reference: 'btnApropos'** Rend un composant adressable depuis le ViewController.
- **this.lookup('btnApropos')** Le récupère - remplace le `down('#id')` du Module 2.
- **handler: 'onApropos'** Une chaîne résout la méthode sur le ViewController.
- **Bénéfice** Découplage et testabilité ; plus de chemins DOM fragiles.

Vue & ViewController

```
// vue
{ xtype: 'button', text: 'À propos',
  reference: 'btnApropos', handler: 'onApropos' }

// ViewController
Ext.define('VIGIE.view.main.MainController', {
  extend: 'Ext.app.ViewController',
  alias: 'controller.main',
  onApropos: function () { /* ... */ }
});
```

Des événements au ViewModel

- **listeners déclaratifs** Les listeners d'un composant pointent une méthode du VC (chaîne).
- **Événement métier** `equipementchoisi` (Module 2) remonte désormais au ViewController.
- **Traduction** Le VC convertit l'événement en mise à jour du ViewModel.
- **Découplage** `fireEvent` côté émetteur, méthode nommée côté VC : zéro couplage direct.

Le VC relaie vers l'état

```
// vue : un enfant publie l'événement
{ xtype: 'arbresites',
  listeners: { equipementchoisi: 'onEquipementChoisi' } }

// ViewController
onEquipementChoisi: function (src, eq) {
  this.getViewModel().set('equipementCourant', eq);
}
```

● BLOC 3.3

ViewModel & databinding

Le ViewModel

- **data** L'état observable de la vue.
- **formulas** Données dérivées, recalculées à la demande.
- **stores** Collections liées aux grilles et listes.
- **Binding** Observe le ViewModel et propage tout changement à l'interface.

Le ViewModel est la source de vérité ; la vue n'en est qu'un reflet.

Le binding : principe

- **bind: '{chemin}'** Lie une config de composant à une donnée du ViewModel.
- **Réactif** La config se met à jour automatiquement quand la donnée change.
- **Chemins imbriqués** {équipementCourant.nom} suit la propriété d'un objet lié.
- **Configs bindables** value, hidden, disabled, title, store... selon le composant.

Publier l'état, le consommer

```
// ViewModel
data: { equipementCourant: null }

// vue : un panneau de détail reflète l'équipement courant
{ xtype: 'displayfield', fieldLabel: 'Équipement',
  bind: '{equipementCourant.nom}' }
```

Dès que le ViewController affecte equipementCourant, le détail se met à jour - sans code de synchronisation.

Données dérivées : formulas

Une formula calcule une valeur à partir d'autres données du ViewModel ; elle n'est recalculée que lorsque ses sources changent.

```
formulas: {  
  enDefault: function (get) {  
    var eq = get('equipementCourant');  
    return eq && eq.get('etat') === 'DEFAULT';  
  }  
}  
  
// vue : { bind: { hidden: '{!enDefault}' } }
```

Binding bidirectionnel

Un champ de saisie lié lit ET écrit la donnée : la modifier dans l'UI met à jour le ViewModel, et inversement.

```
// ViewModel
data: { seuil: 80 }

// vue : le champ publie ses modifications dans {seuil}
{ xtype: 'numberfield', fieldLabel: 'Seuil',
  bind: '{seuil}' }
```

Un store dans le ViewModel

Le ViewModel peut héberger un store, lié ensuite à une grille. Données en dur ici ; au Module 4, un proxy les remplacera.

```
stores: {  
  equipments: {  
    fields: ['nom', 'etat'],  
    data: [ { nom: 'Pompe P-12', etat: 'OK' } ]  
  }  
}  
  
// vue : { xtype: 'grid', bind: { store: '{equipements}' } }
```

C'est la méthode recommandée pour lier une grille à des données dans une architecture MVVM, plutôt que de définir le store de manière globale.

Quand binder, quand ne pas le faire

Binder

- État partagé entre composants, maître-détail.
- UI dérivée de données (visibilité, libellés).
- Activation/désactivation conditionnelle.

Préférer l'impératif

- Action ponctuelle one-shot (pas un état).
- Transformation lourde réévaluée souvent (formula coûteuse).
- Forte fréquence de changements (churn de notifications).

Mesurer le coût des formules et le churn sur des cas réels - pas de binding par dogme.

● LAB M3

Refactor MVVM

Refactor MVVM & maître-détail

SOCLE - POUR TOUS

- Extraire les handlers inline vers le MainController.
- Déclarer un MainModel avec equipementCourant.
- Lier un panneau de détail par binding.
- Relayer equipementchoisi du VC vers le VM.

EXTENSION - POUR ALLER PLUS LOIN

- Une formula enDefault pilotant la visibilité.
- Un champ « seuil » en binding bidirectionnel.
- Mesurer l'effet d'une formula sur un changement fréquent.

État attendu en fin de lab

- VIGIE est refactorée : vue déclarative, MainController, MainModel.
- Sélectionner un équipement met à jour le panneau de détail par binding.
- Plus aucune logique métier ni handler inline dans la vue.

Resynchro : dépôt VIGIE de référence (état fin M3) fourni.

Ce qui est acquis, ce qui vient

Acquis - M3

- V / VC / VM.
- reference / lookup.
- Databinding & formulas.
- Maître-détail de VIGIE.

À venir - M4

- Models, fields, validations.
- Associations & proxies.
- Stores distants & Promises.
- VIGIE sur une API mock.

● FIN DU Module 3

Place au Lab M3

Prochaine étape : Couche données (Module 4).



VIGIE

Couche données

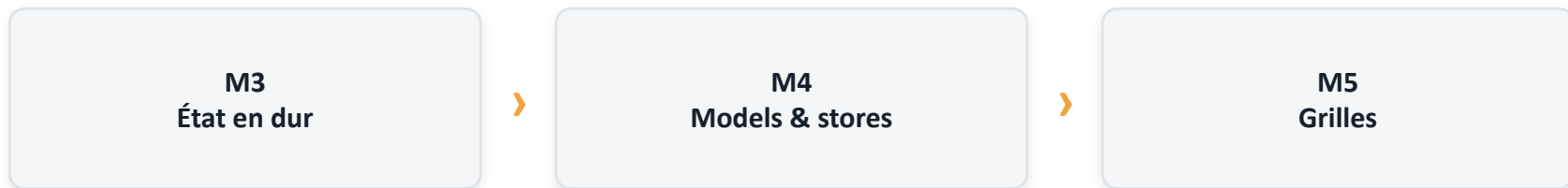
ExtJS 8 · Module 4

Au programme

- **Bloc 4.1 - Models** Fields, validations, associations.
- **Bloc 4.2 - Proxies & chargement** ajax/rest, readers, Promises.
- **Bloc 4.3 - Stores** Chargement, tri, filtrage, chained stores.
- **Lab M4 - VIGIE sur l'API mock** Brancher le maître-détail sur des données réelles.

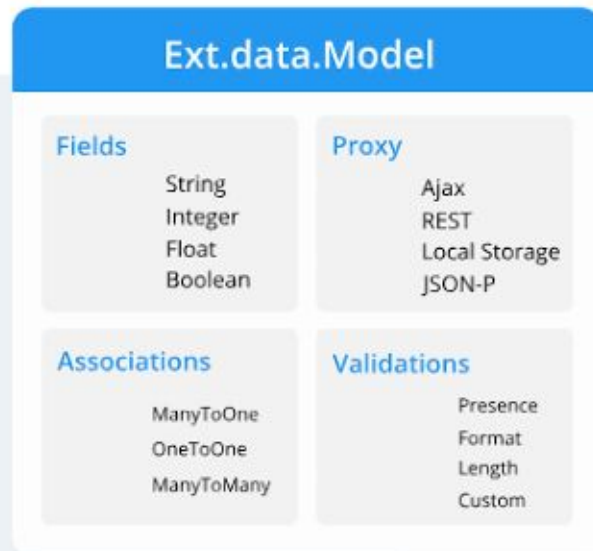
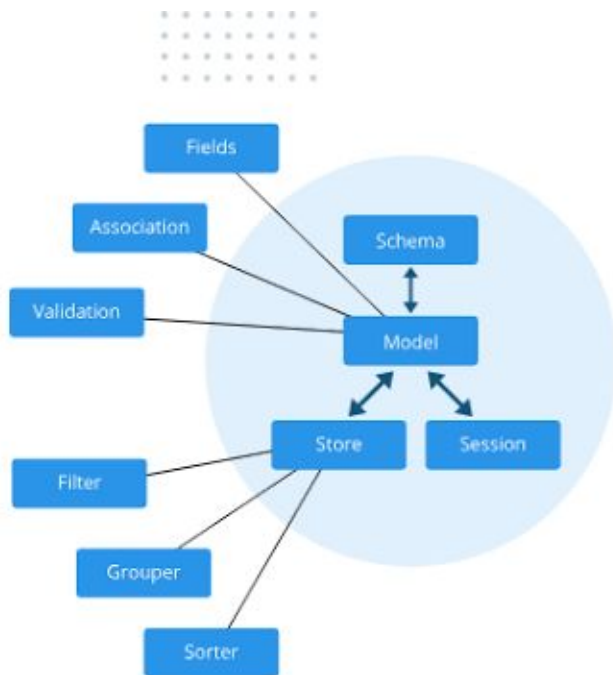
En finir avec les données en dur

Le maître-détail du Module 3 fonctionne sur des objets fabriqués à la main. Pour une vraie application, les données viennent d'un serveur.



Objectif : modéliser le domaine et l'alimenter depuis une API, puis rebrancher le binding dessus.

Modèle de données



À l'issue de cette partie

- 1 Modéliser le domaine : fields typés, validations, associations.
- 2 Charger et écrire via proxies et readers ; gérer l'asynchrone par Promises.
- 3 Alimenter VIGIE par des stores : tri, filtrage, chained stores.
- 4 Rebrancher le maître-détail sur l'API mock.

● BLOC 4.1

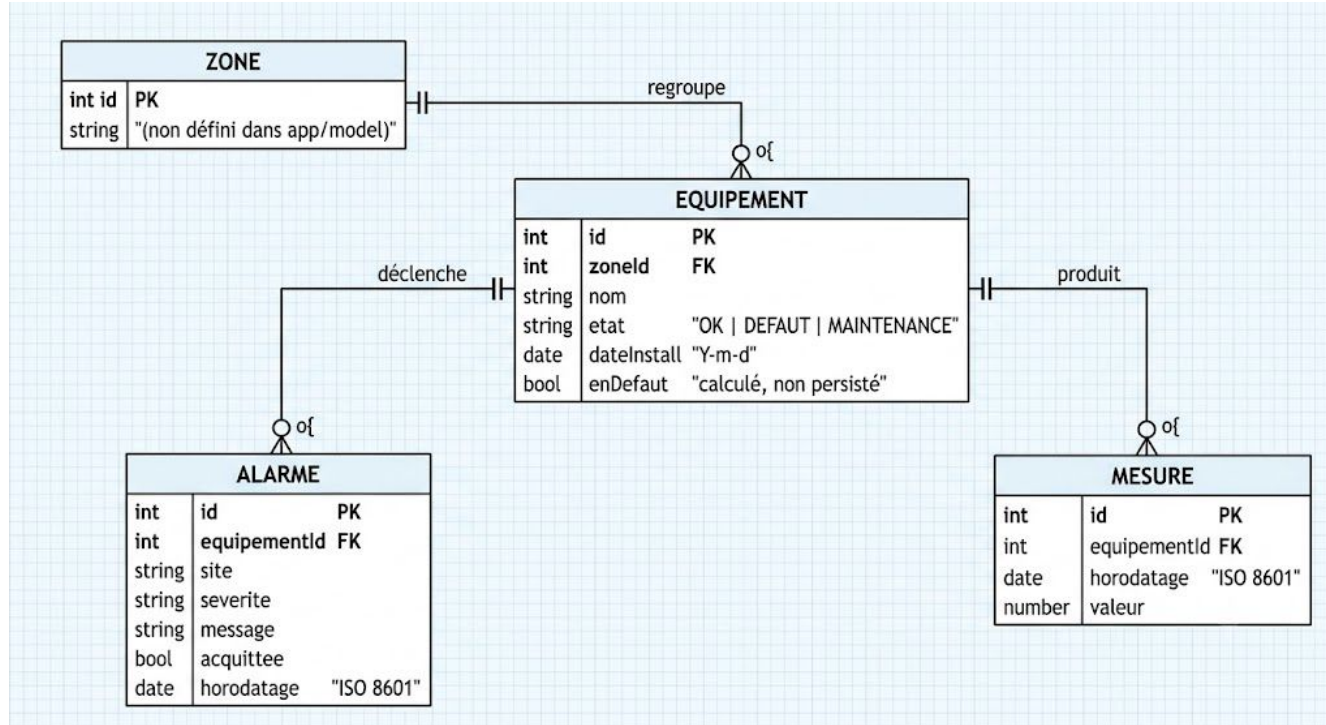
Models

Le Model : forme & règles d'un enregistrement

- **fields** Décrivent les attributs et leur type (int, string, date, boolean).
- **Champ calculé** calculate dérive une valeur des autres champs.
- **Conversion** Le type normalise la valeur reçue du serveur.
- **idProperty** Identifie l'enregistrement (par défaut id) pour le suivi et l'écriture. Sans cela, ExtJS ne sait pas distinguer deux enregistrements et le CRUD ne fonctionnera pas correctement.

À partir d'ici, l'équipement devient un véritable Ext.data.Model (il était une classe à config system au Lab M1).

Modèle de données



Définir le Model Equipement

```
Ext.define('VIGIE.model.Equipement', {  
  extend: 'Ext.data.Model',  
  fields: [  
    { name: 'id', type: 'int' },  
    { name: 'nom', type: 'string' },  
    { name: 'etat', type: 'string' },  
    { name: 'enDefaut', calculate: function (d) {  
      return d.etat === 'DEFAULT';  
    } }  
  ]  
});
```

Validations

Les validators attachent des règles aux champs. `isValid()` vérifie l'enregistrement ; `getErrors()` détaille les manquements.

```
validators: {  
  nom: 'presence',  
  etat: { type: 'inclusion',  
    list: ['OK', 'DEFAULT', 'MAINTENANCE'] }  
}  
  
eq.isValid();    // false si etat hors liste
```

Associations

- **reference** Un champ clé étrangère déclare le lien vers le Model parent.
- **Hiérarchie VIGIE** Site → Zone → Équipement ; Équipement → Alarmes.
- **Navigation générée** `zone.getSite()` ; `equipement.alarmes()` (store lié).
- **Cohérence** Les enregistrements liés se chargent et se suivent ensemble.

Déclarer les liens

```
// Zone → Site
{ name: 'siteId',      reference: 'Site' }

// Alarme → Equipement (génère equipement.alarmes())
{ name: 'equipementId', reference: 'Equipement' }

zone.getSite();           // l'enregistrement parent
equipement.alarmes();     // le store des alarmes liées
```

● BLOC 4.2

Proxies & chargement

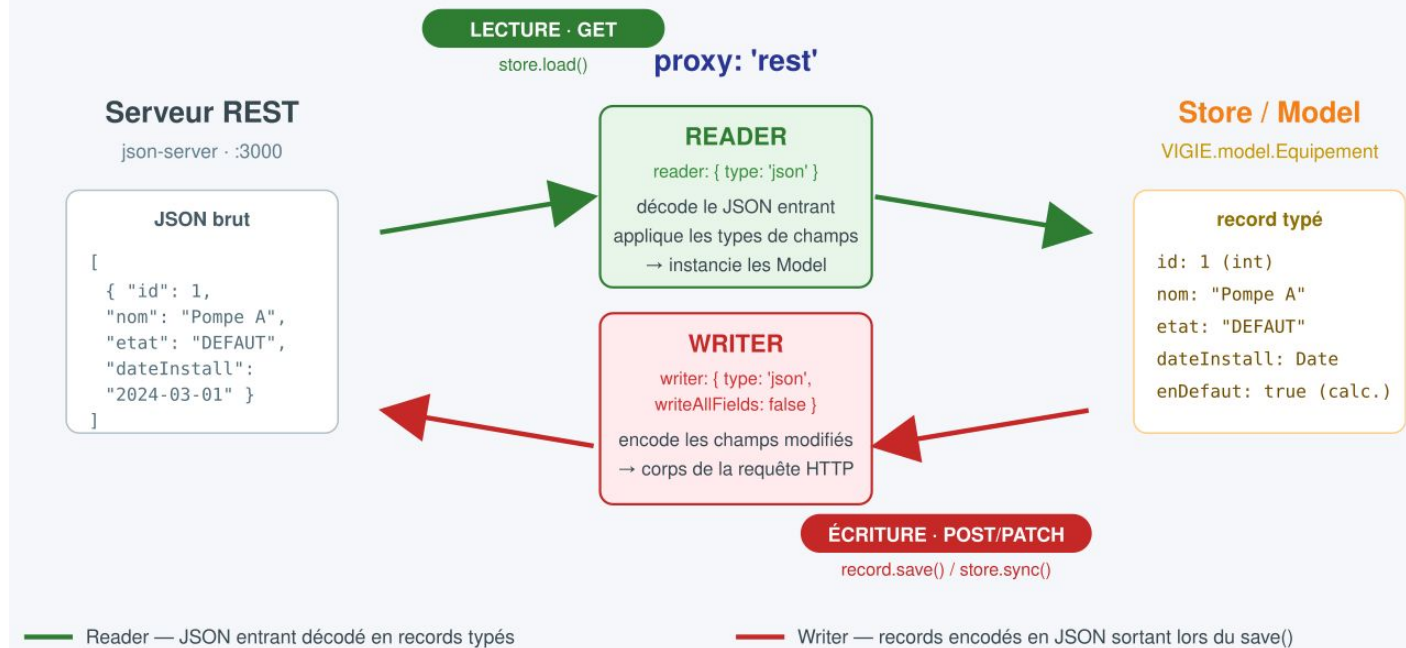
Le proxy : pont vers le serveur

- **ajax** Lecture/écriture sur des URL libres (verbe paramétrable).
- **rest** Convention REST : GET/POST/PUT/DELETE déduits de l'opération.
- **memory** Données locales en mémoire - utile pour prototyper.
- **reader / writer** Transforment le JSON entrant/sortant (rootProperty, mapping).

Le proxy : pont vers le serveur

VIGIE - Reader & Writer dans le Proxy REST

Ext.data.proxy.Rest - sérialisation JSON entre le serveur et le Store/Model



Brancher un proxy sur le Model

```
proxy: {  
  type: 'rest',  
  url:  '/api/equipements',  
  reader: {  
    type: 'json',  
    rootProperty: 'data'  
  }  
}
```

Le format concret du mock (JSON statique ou serveur) est un arbitrage à fixer avant le lab.

Chargement asynchrone & Promises

Le chargement est asynchrone. Une Promise enchaîne succès et échec proprement, sans imbriquer les callbacks.

```
Ext.Ajax.request({ url: '/api/equipements' })
  .then(function (response) {
    var data = Ext.decode(response.responseText);
    // alimenter le store...
  })
  .catch(function (err) { /* gérer l'échec */ });
```

● BLOC 4.3

Stores

Le store : une collection vivante

- **model + proxy** Le store charge des enregistrements typés via le proxy.
- **autoLoad** Déclenche le chargement initial à la création.
- **sorters / filters** Tri et filtrage, applicables et révocables à la volée.
- **Événements** load, datachanged, update - observés par l'UI liée.

Définir & exploiter un store

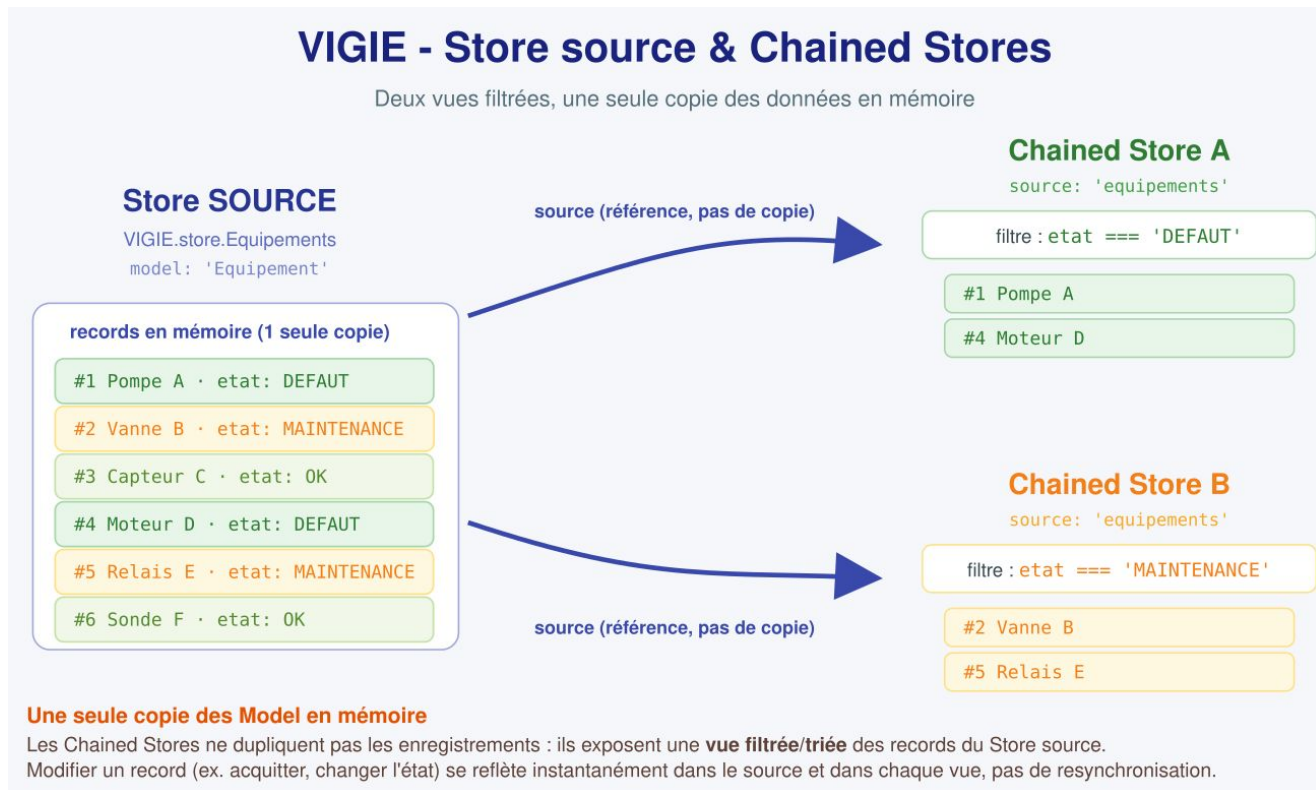
```
Ext.define('VIGIE.store.Equipements', {  
    extend: 'Ext.data.Store',  
    alias: 'store.equipements',  
    model: 'VIGIE.model.Equipement',  
    autoLoad: true  
});  
  
store.filter('etat', 'DEFAULT');  
store.load({ callback: function (recs, op, ok) {} });
```

Chained stores : une vue dérivée

Un chained store partage les enregistrements d'un store source, mais applique son propre tri/filtre - sans recharger.

```
Ext.define('VIGIE.store.AlarmesActives', {  
    extend: 'Ext.data.ChainedStore',  
    source: 'alarmes',  
    filters: [{ property: 'acquittee', value: false }]  
});
```

Chained stores : une vue dérivée



Trier & filtrer : client ou serveur ?

Côté client

- Petits volumes déjà chargés.
- Tri/filtre instantanés, sans aller-retour.
- Idéal pour des listes courtes.

Côté serveur

- Gros volumes : pagination (pageSize).
- remoteSort / remoteFilter délèguent au backend.
- Une page à la fois, charge maîtrisée.

Mesurer le volume réel : ne pas charger 100 000 lignes pour en filtrer 20.

Rebrancher le binding sur des données réelles

Le store du ViewModel n'est plus en dur : il référence le store défini, alimenté par le proxy. Le binding du Module 3 ne change pas.

```
// MainModel
stores: {
  equipments: { type: 'equipements' }
}

// vue : { xtype: 'grid', bind: { store: '{equipements}' } }
```

Sencha Cmd au quotidien

- **sencha app watch** Build incrémental + rechargement pendant le développement.
- **sencha app build** Bundle development ou production (détaillé au Module 8).
- **Workspaces** Plusieurs apps/packages partageant un même ext/ (mention).
- **URL du mock** En dev, pointer le proxy vers l'API mock retenue.

● LAB M4

VIGIE sur l'API mock

VIGIE sur l'API mock

SOCLE - POUR TOUS

- Définir le Model Equipement (fields + validations).
- Brancher un proxy sur l'API mock.
- Définir le store Equipements (autoLoad).
- Rebrancher le maître-détail : binding direct.

EXTENSION - POUR ALLER PLUS LOIN

- Associations Équipement → Alarmes.
- Un chained store « alarmes actives ».
- Chargement par Promise avec gestion d'échec.

État attendu en fin de lab

- Equipement est un Ext.data.Model, branché sur l'API mock.
- Le store alimente VIGIE ; le maître-détail lit des données réelles.
- Le binding direct {equipementCourant.nom} remplace les formules du Module 3.

Resynchro : dépôt VIGIE de référence (état fin M4) fourni, mock inclus.

Ce qui est acquis, ce qui vient

Acquis - M4

- Models, fields, validations.
- Associations & proxies.
- Stores, filters, chained.
- VIGIE sur l'API mock.

À venir - M5

- Grilles : édition, groupes.
- Pivot, export, sélection.
- Performance : buffered rendering.
- Journal d'alarmes de VIGIE.

● FIN DU Module 4

Place au Lab M4

Prochaine étape : Grilles (Module 5).



VIGIE

Grilles

ExtJS 8 · Module 5

Au programme

- **Bloc 5.1 - Grille avancée** Renderers, sélection, édition, groupes, drag-drop.
- **Bloc 5.2 - Grilles spécialisées** Pivot, sélection tableur, export.
- **Bloc 5.3 - Performance** Le point délibéré : buffered rendering & store.
- **Lab M5 - Journal d'alarmes** La grille de VIGIE, complète et performante.

La grille brute ne suffit plus

Le Module 4 a produit une grille à deux colonnes. Le journal d'alarmes de VIGIE exige bien davantage : édition, groupes, export, et tenue de charge.



Objectif : transformer cette grille en outil de supervision, sans sacrifier la performance.

À l'issue de cette partie

- 1 Enrichir la grille : renderers, sélection, édition, groupes, drag-drop.
- 2 Exploiter les grilles spécialisées : pivot, sélection tableur, export.
- 3 Diagnostiquer et traiter la performance : buffered rendering & store.
- 4 Bâtir le journal d'alarmes de VIGIE.

● BLOC 5.1

Grille avancée

Colonnes & renderers

- **dataIndex** Lie la colonne à un field du Model.
- **renderer** Transforme la valeur en contenu affiché (texte, HTML, couleur).
- **Colonnes typées** datecolumn, numbercolumn, checkcolumn, widgetcolumn.
- **Prudence** Le renderer est appelé pour chaque cellule visible - le garder léger (cf. perf).

Colonnes & renderers

Application Users			
Name	Email Address	Phone Number	Birth Date
Lisa	lisa@simpsons.com	555-111-1224	10/28/1981
Bart	bart@simpsons.com	555-222-1234	
Homer	home@simpsons.com	555-222-1244	
Marge	marge@simpsons.com	555-222-1254	

October 1981

S	M	T	W	T	F	S
27	28	29	30	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
1	2	3	4	5	6	7

Today

Un renderer de sévérité

```
{ text: 'Sévérité', dataIndex: 'severite',  
  renderer: function (v) {  
    var c = v === 'haute' ? '#E5534B' : '#5B6876';  
    return '<span style="color:' + c +  
      '">' + v + '</span>';  
  }  
}
```

Préférer une classe CSS à du style inline calculé quand le volume grandit.

Sélection & édition

- **Modèles de sélection** rowmodel, cellmodel, checkboxmodel, spreadsheet.
- **Cell editing** Édition d'une cellule à la fois ; rapide pour des corrections ponctuelles.
- **Row editing** Édition d'une ligne entière en bloc, validée d'un coup.
- **editor** La config editor d'une colonne déclare le champ de saisie.

Éditer l'acquittement

```
plugins: [{ ptype: 'cellediting', clicksToEdit: 1 }],
columns: [
  { text: 'Message', dataIndex: 'message', flex: 1 },
  { xtype: 'checkcolumn', text: 'Acquittée',
    dataIndex: 'acquittee' }
]
```

La modification est écrite dans le Model, puis poussée au serveur via le proxy rest (Module 4).

Groupes repliables

La feature grouping regroupe les lignes par champ ; les en-têtes sont repliables et peuvent afficher un décompte.

```
features: [{ ftype: 'grouping',  
  collapsible: true,  
  groupHeaderTpl: '{name} ({rows.length})' }],  
  
// store  
store: { groupField: 'site' }
```

Groupes repliables

Orders

Order Id	Amount	Date
O12	1000	29/05/2015

Réorganiser par glisser-déposer

Le plugin `gridviewdragdrop` autorise le déplacement de lignes ; l'ordre du store est mis à jour en conséquence.

```
plugins: [{  
  ptype: 'gridviewdragdrop',  
  dragText: 'Déplacer {0} ligne(s)'  
}]
```

À réserver à un ordre métier explicite ; sur un tri serveur, le drag-drop n'a pas de sens.

● BLOC 5.2

Grilles spécialisées

Pivot & sélection tableur

- **Pivot Grid** Tableau croisé dynamique (package pivot) : agrégats par dimensions.
- **Cas VIGIE** Alarmes par site × sévérité, comptées ou moyennées.
- **Spreadsheet selection** Sélection façon tableur : lignes, colonnes, plages, copier.
- **À situer** Puissant mais lourd : à réserver à un vrai besoin analytique.

Exporter une grille

Le plugin gridexporter sérialise la grille en Excel ou CSV, côté client, sans aller-retour serveur.

```
plugins: 'gridexporter',  
  
// déclencher l'export  
grid.saveDocumentAs({  
  type: 'xlsx', title: 'Alarmes',  
  fileName: 'alarmes.xlsx'  
});
```

● BLOC 5.3

Performance

D'où vient la lenteur d'une grille

- **Le DOM** Matérialiser 50 000 lignes sature le navigateur - c'est le coût dominant.
- **Les renderers** Appelés par cellule visible, à chaque rafraîchissement.
- **Le layout** Des recalculs en cascade pendant le scroll (thrashing).
- **Le volume chargé** Tout charger d'un coup gèle l'interface au démarrage.

Buffered rendering

La grille ne matérialise dans le DOM que les lignes visibles, plus une marge. Actif par défaut ; la marge se règle.

```
// Ext.grid.Panel - rendu bufferisé actif par défaut
plugins: {
  bufferedrenderer: {
    leadingBufferZone: 20,
    trailingBufferZone: 10
  }
}
```

Buffered rendering : Illustration

My Grid Panel											
Id	FirstName	MiddleName	LastName	Age	Country	City	Street	Mobile	Phone	Zip	
1	cmEzvrSb	DYqIh	XGridQTPEe	19	keLmwHS3tZ	CMdRPkvcOZ	cdyNuGV	1401588147	6745605	52883	
2	V5WwUDIq	UWTib	IZHeboASmc	83	rDqgGeLUnu	IcfIqCvXBa	FcyhRTr	1350251232	7217712	37792	
3	VZYnBI	XqslJe	VmhYKJCPsU	32	dWaeINBbFP	LSZvduAaIiz	Kotzlau	324588725	9835205	51797	
4	LDiFowSM	QzZno	bolHvyDxkF	19	hPzvBoXEDi	oytNOLPRhz	piWqBZxN	244377485	9080810	92007	
5	STBIOxip	TLXzB	MujDURcCK	9	RIPBcgCOq	piFzmbvOga	mRnVhUE	1177306503	2747802	14028	
6	rDRvICAY	wWuP	IEDYcVoTj	76	ZAQcgBTvJo	DHJxYybinq	ngxwioI	1121967632	3407592	80075	
7	ovBpFRMc	IYHad	vspPuGbMkq	65	OULyQqVhNR	JSIhdvOPnb	grqTOfc	628522196	4936218	82214	
8	ITFtpcEY	aRoSf	JzdakGHBOs	81	JjvcMxJUpn	ImJBNHRns	SNBFPAY	657396522	6162719	26913	
9	ZyXAReKM	hlerf	QLskJnfdA	73	hOWHBgJMxm	ZRpfeFHMQ	NpHvtIy	297521735	5943298	73681	
10	YVlouqZp	FAUQN	SqdpQdJlf	33	ICbBtwRfVTV	RYUVTKdJN	xfkWinj	156979938	5115051	86517	
11	xHtAOMIN	ySKNa	fXGBIFJIsU	40	MKFfqQcApT	IERdjVGSkB	hQJFSnt	279362323	343322	66583	
12	icBQWhe	mCZkV	rIHeWSmSb	47	dkozUVOuH	NsqTUBnMI	IOylwTX	569181370	5783691	29397	
13	ixFvIqTX	EXIOt	jBTmWodFUO	64	XoEAlqwDWg	fsZqhykmC	wFyuVma	666734418	5724792	4260	
14	ICpUJvWt	uYtOm	bxOJcbdjFE	44	BcmdJbwfCW	UDBGOruRg	KctfJT	889940268	6547241	27291	
15	exgvmQn	pkSf	PgBRyOZuaF	73	PrTvDpmXY	MPmzBWiWdXj	FoJwAt	1306057592	4774169	47769	
16	rwMFyBOT	jrXnz	mJRgxvhZCX	39	SMDpyoInK	OLmzMSxJRh	ZufzjON	470810718	9429321	62509	
17	RdnSOQv	jHlIc	YDESZmMsJ	75	yZHeiamxdh	vSIYBLUFxK	DomhBW	155645952	5177612	46896	
18	zOYnJgQ	dILis	TILVJdtku	42	XcwJDudRx	RVKtDKBy	KnjXcZF	326998505	2947998	7241	
19	PWVcGySq	ayVWJ	eEmodZJuxR	51	WZAPEpJMK	eBsyUXWCxE	xHdmQfr	1327487419	6027221	22970	
20	mexRkYdr	DgRpo	UFHhIGBDx	13	ntcbPhxYVWy	QqWopMeagO	wFVnxcq	977897228	164184	1010	
21	kSPbAYN	hOcxR	PlspcvZxnr	11	IvfcPBktj	YZufnPhyU	uAexVgr	626628798	5347595	74365	
22	fWRzEecJ	BXMPd	YrhCgvwdDT	83	kuKCTalVtmA	hBRgQVYtby	RuVnJUB	993646860	2651367	52664	
23	JuUmeRwI	lbaeZ	WhqJfHMeZ	31	oZGzWSKcmM	yBePLbAotI	WNEmDUF	972819478	7176513	66323	
24	FxrUitDh	StAlm	JLhBIZdmZ	0	FysztQGEW	umyjdahHbS	yFszKmo	1317374951	6178894	74230	
25	IxmPRyi	uANzf	qGXstohIEC	2	VkaTROGQCn	HSsoEMfVgc	AZcubgl	703440528	2067871	4153	
26	RvccqTJZN	kgPBz	ajBacALmq	41	dxVQBgpgFH	cOrQAVzINDR	ailBgCc	710540772	3025207	8587	
27	zQqgYyJO	Zaunp	nivhsxHUBE	51	mkQoYDAGX	EzZSnFskKQ	XSUNTai	765363261	8841247	94946	
28	efhObacz	DdQwX	BPOqMkKOYT	36	gZdDmAOmb	jZobhQmpwv	jdIiqm	135076761	6464843	84353	
29	oCMwecfd	uIChE	TobhBCLukZ	51	AuwitbHdTh	HVaeCRaHdEs	oQhobLO	433373068	8027001	11856	

Viewport

Buffered store : charger par pages

Le store bufferisé ne charge que les pages nécessaires au défilement. Le proxy doit paginer (le mock json-server le sait).

```
Ext.create('Ext.data.BufferedStore', {  
    model: 'VIGIE.model.Alarme',  
    pageSize: 100,  
    autoLoad: true  
});
```

Couplé au buffered rendering, il rend une grille de 50 000 lignes fluide.

Renderers coûteux & layout thrashing

- **Renderer léger** Pas de requête DOM, pas de calcul lourd, pas de création d'objet.
- **CSS plutôt qu'inline** Une classe vaut mieux qu'un style recalculé à chaque cellule.
- **Éviter le thrashing** Ne pas déclencher de mesure/layout pendant le défilement.
- **widgetcolumn** Puissant mais coûteux : à réserver aux colonnes qui le justifient.

Mesurer avant d'optimiser

D'abord mesurer

- Temps de rendu, nombre de lignes réel.
- Profil navigateur (scripting / rendering).
- Identifier la cause dominante.

Ensuite cibler

- Volume → buffered store + pagination.
- DOM → buffered rendering.
- Rendre → l'alléger ou passer en CSS.

Sur des mesures et des cas réels, pas sur du dogme : optimiser la cause, pas au hasard.

● LAB M5

Journal d'alarmes

Journal d'alarmes

SOCLE - POUR TOUS

- Grille d'alarmes avec renderer de sévérité.
- Groupes repliables par site.
- Édition de l'acquittement (écriture proxy).
- Export Excel de la grille.

EXTENSION - POUR ALLER PLUS LOIN

- Buffered store + rendering sur 50 000 alarmes.
- Mesurer le gain (avant / après).
- Optimiser un renderer coûteux.

État attendu en fin de lab

- Le journal d'alarmes affiche, groupe et édite l'acquittement (écriture serveur).
- L'export Excel fonctionne depuis la grille.
- La grille reste fluide sur un gros volume (buffered).

Resynchro : dépôt VIGIE de référence (état fin M5) fourni.

Ce qui est acquis, ce qui vient

Acquis - M5

- Renderers, édition, groupes.
- Drag-drop, pivot, export.
- Sélection tableur.
- Performance : buffered.

À venir - M6

- Arbres & fil d'Ariane.
- Formulaires liés au VM.
- Widgets & charts.
- Dashboard de VIGIE.

● FIN DU Module 5

Place au Lab M5

Prochaine étape : Arbres, formulaires, widgets & charts (Module 6).



VIGIE

Arbres, formulaires & charts

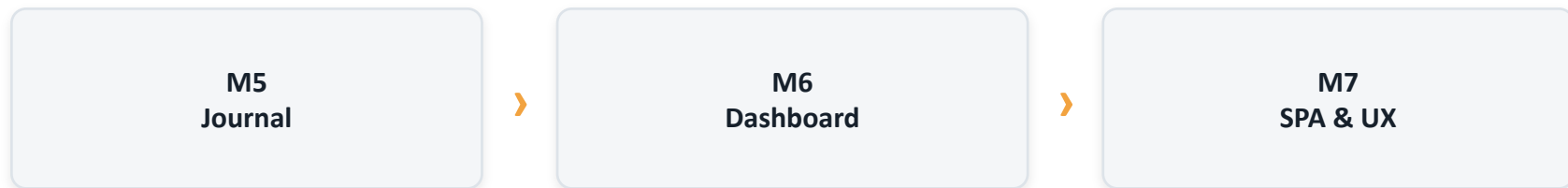
ExtJS 8 · Module 6

Au programme

- **Bloc 6.1 - Arbres & navigation** TreeStore, fil d'Ariane, Tree List, drag-drop.
- **Bloc 6.2 - Formulaires liés & widgets** Champs liés au VM, validations, widgetcolumn.
- **Bloc 6.3 - Charts** Télémétrie : séries, axes, interactions.
- **Lab M6 - Dashboard** Arbre + journal filtré + chart, liés par sélection.

Il manque la navigation et la vue d'ensemble

VIGIE sait charger et éditer ses données. Mais on ne peut ni naviguer dans la hiérarchie des sites, ni visualiser la télémétrie.



Objectif : assembler arbre, journal et charts en un dashboard piloté par la sélection.

À l'issue de cette partie

- 1 Construire la navigation : arbre Site→Zone→Équipement + fil d'Ariane.
- 2 Lier un formulaire de configuration au ViewModel (validations, soumission).
- 3 Visualiser la télémétrie par des charts.
- 4 Assembler le dashboard de VIGIE.

● BLOC 6.1

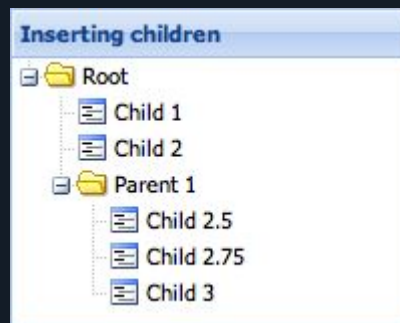
Arbres & navigation

L'arbre : une hiérarchie navigable

- **TreeStore** Un store de nœuds parents/enfants - données imbriquées ou chargées à la demande.
- **treepanel** Affiche l'arbre ; rootVisible masque la racine technique.
- **Hiérarchie VIGIE** Site → Zone → Équipement, reflétant les associations du Module 4.
- **leaf / children** Un nœud feuille n'a pas d'enfants ; les autres se déploient.

Définir l'arbre des sites

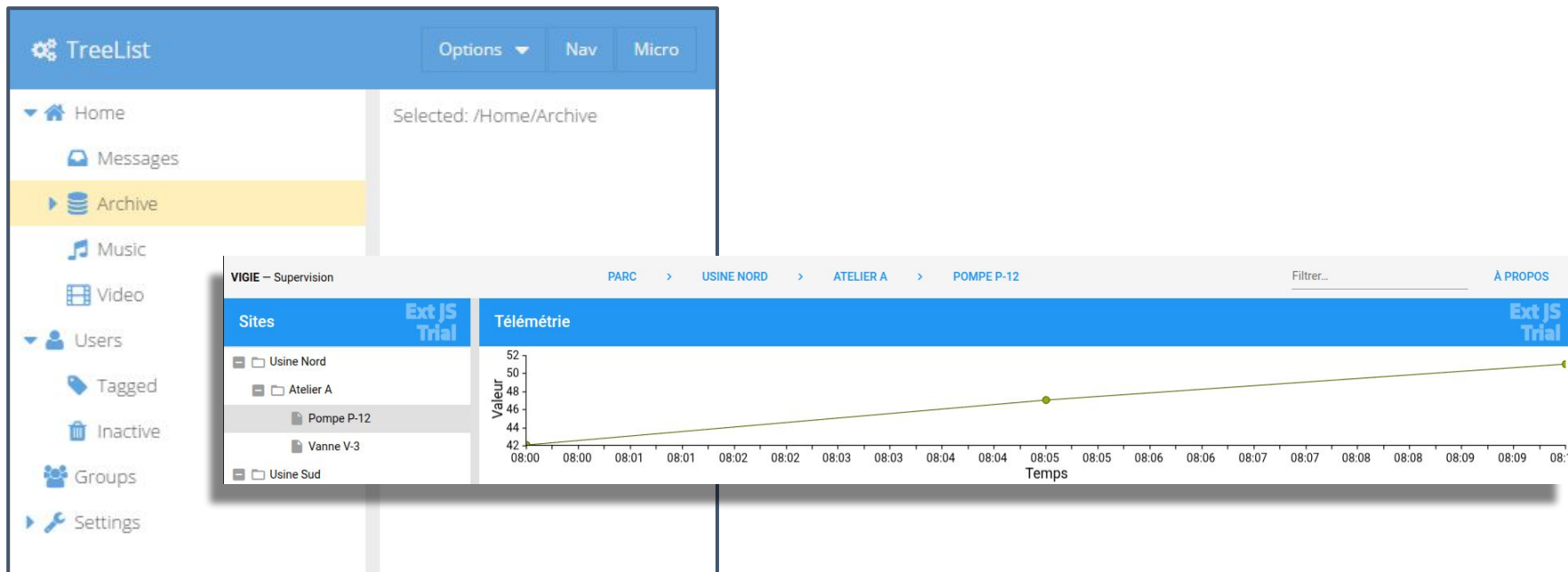
```
{ xtype: 'treepanel', title: 'Sites', rootVisible: false,  
  store: {  
    root: { expanded: true, children: [  
      { text: 'Usine Nord', children: [  
        { text: 'Atelier A', children: [  
          { text: 'Pompe P-12', leaf: true }  
        ] }  
      ] }  
    ] }  
  }  
}
```



Fil d'Ariane & Tree List

- **breadcrumb** Une barre de fil d'Ariane liée au TreeStore : chemin courant et navigation.
- **Synchronisation** Sélectionner dans l'arbre met à jour le fil d'Ariane, et inversement.
- **Tree List** Navigation compacte (treelist), idéale en barre latérale, avec mode réduit.
- **Quand l'un, quand l'autre** treepanel pour explorer en profondeur ; treelist pour naviguer une app.

Fil d'Ariane & Tree List : Illustration



Réorganiser l'arbre

Le plugin `treeviewdragdrop` autorise le déplacement de nœuds ; la structure du `TreeStore` est mise à jour.

```
viewConfig: {  
  plugins: {  
    treeviewdragdrop: { appendOnly: false }  
  }  
}
```

À réserver à une réorganisation métier réelle ; persister les déplacements via le proxy.

L'arbre pilote le dashboard

- **Sélection** → **état** Choisir un nœud publie l'équipement (ou le site) courant au ViewModel.
- **Réutilisation** Le même mécanisme équipementchoisi posé au Module 2 sert ici de pivot.
- **Journal filtré** Le store des alarmes se filtre sur le site/équipement sélectionné.
- **Détail & chart** Le panneau de détail et le chart de télémétrie suivent par binding.

● BLOC 6.2

Formulaires liés & widgets

Formulaires liés au ViewModel

- **Champs liés** bind sur un field de l'enregistrement courant - édition bidirectionnelle.
- **Validations** allowBlank, vtype, ou les validators du Model remontés à l'affichage.
- **État du formulaire** isValid() conditionne l'activation du bouton d'enregistrement.
- **Pas de logique dans la vue** La soumission est orchestrée par le ViewController.

Un formulaire de configuration

```
{ xtype: 'form', items: [
  { xtype: 'textfield', fieldLabel: 'Nom',
    bind: '{équipementCourant.nom}', allowBlank: false },
  { xtype: 'combobox', fieldLabel: 'État',
    bind: '{équipementCourant.etat}',
    store: ['OK', 'DEFAULT', 'MAINTENANCE'] }
] }
```

Le champ écrit directement dans l'enregistrement lié - qui devient « dirty ».

Un formulaire de configuration : Illustration

The image shows a web form titled "User Form" with three input fields: "First Name:", "Last Name:", and "Date of Birth:". The "Date of Birth:" field contains the text "bob" and is surrounded by a red dashed border, indicating a validation error. Below this field, a red error message reads: "!" "bob" bad. "m/d/Y" good.

User Form

First Name:

Last Name:

Date of Birth:

! "bob" bad. "m/d/Y" good.

Enregistrer les modifications

L'enregistrement lié est modifié en place. Le persister revient à le sauvegarder via son proxy - pas de `form.submit()` vers une URL ad hoc.

```
// ViewController - handler du bouton « Enregistrer »  
onEnregistrer: function () {  
    var eq = this.getViewModel().get('equipementCourant');  
    if (eq.isValid()) eq.save();  
}
```

Widgets dans une grille

Une widgetcolumn rend un composant vivant par ligne (barre de progression, jauge, bouton). Puissant, mais à doser (cf. perf, Module 5).

```
{ xtype: 'widgetcolumn', text: 'Niveau',  
  dataIndex: 'niveau',  
  widget: { xtype: 'progressbarwidget',  
            textTpl: '{value:percent}' }  
}
```

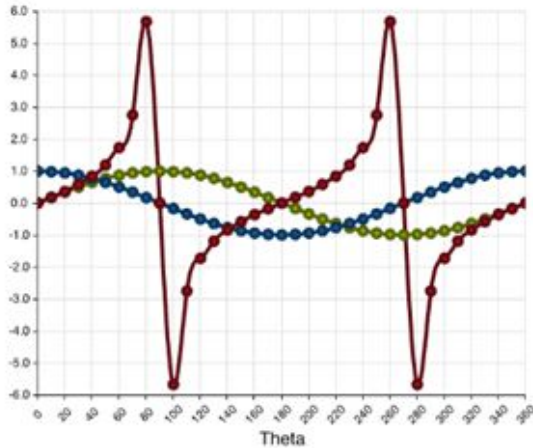
● BLOC 6.3

Charts

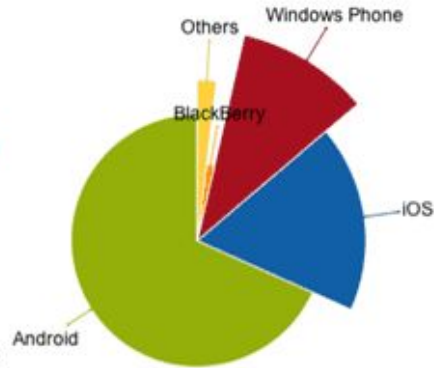
Anatomie d'un chart

- **store** Le chart lit ses données dans un store, comme une grille.
- **axes** numeric, time, category - définissent l'échelle de chaque dimension.
- **series** line, bar, area, pie... reliées à des xField / yField.
- **Package charts** Les charts viennent du package sencha-charts (à requérir).

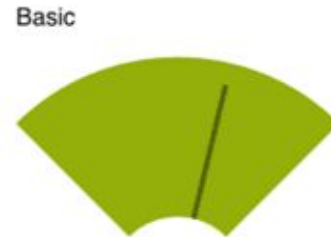
Charts : Exemples



Cartesian Chart



Polar Chart



SpaceFilling Chart

Un chart de télémétrie

```
{ xtype: 'cartesian', store: { type: 'mesures' },  
  axes: [  
    { type: 'numeric', position: 'left', title: 'Valeur' },  
    { type: 'time',    position: 'bottom', title: 'Temps' }  
  ],  
  series: [{ type: 'line',  
    xField: 'horodatage', yField: 'valeur' }]  
}
```

Rendre le chart exploitable

- **interactions** panzoom pour explorer une période, itemhighlight pour pointer une valeur.
- **tooltip** Afficher la valeur exacte au survol - le détail que le tracé n'offre pas.
- **Rafraîchissement** Lié au store : une nouvelle mesure met le tracé à jour.
- **Lisibilité** Trop de séries tue la lecture ; séparer plutôt que superposer.

Visualiser ou tabuler ?

Un graphique

- Tendence dans le temps, saisonnalité.
- Comparaison de magnitudes, distribution.
- Repérer une anomalie d'un coup d'œil.

Un tableau

- Valeurs exactes à lire ou comparer.
- Tri, filtrage, édition ligne à ligne.
- Export et traçabilité.

Un chart ne remplace pas un tableau : il répond à une autre question. Le dashboard combine les deux.

● LAB M6

Dashboard

Dashboard de VIGIE

SOCLE - POUR TOUS

- Arbre Site→Zone→Équipement + fil d'Ariane.
- Sélection d'un nœud → journal filtré.
- Chart de télémétrie lié à l'équipement.
- Assembler arbre + journal + chart.

EXTENSION - POUR ALLER PLUS LOIN

- Drag-drop de réorganisation dans l'arbre.
- Formulaire de configuration lié au VM.
- Validation + bouton « Enregistrer » conditionnel.

CHECKPOINT

État attendu en fin de lab

- Le dashboard assemble arbre, journal et chart de télémétrie.
- Sélectionner un nœud filtre le journal et met à jour le chart.
- Le fil d'Ariane reflète la position courante dans la hiérarchie.

***Resynchro** : dépôt VIGIE de référence (état fin M6) fourni.*

Ce qui est acquis, ce qui vient

Acquis - M6

- Arbres & fil d'Ariane.
- Formulaires liés & validations.
- Widgetcolumn.
- Charts & dashboard.

À venir - M7

- Routes & deep linking.
- Responsive (toolkits, config).
- Thèmes (Fashion, Triton).
- Internationalisation.

● FIN DU Module 6

Place au Lab M6

Prochaine étape : SPA & UX (Module 7).



VIGIE

SPA & UX

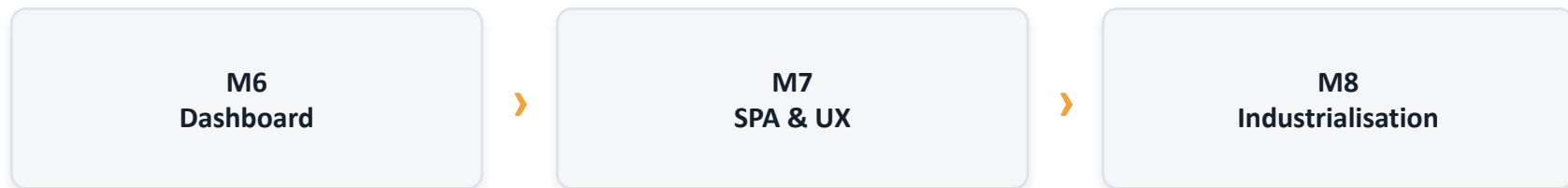
ExtJS 8 · Module 7

Au programme

- **Bloc 7.1 - Routes & SPA** Navigation par URL, deep linking.
- **Bloc 7.2 - Responsive** Toolkits, responsiveConfig, colonnes adaptatives.
- **Bloc 7.3 - Thèmes** Fashion, variables Sass, thème de marque.
- **Bloc 7.4 - Internationalisation** Locale & libellés externalisés.

Un dashboard, mais figé

VIGIE affiche tout sur une seule URL, en français, dans une mise en page fixe. Il manque la navigabilité, l'adaptabilité, la marque et les langues.



Objectif : rendre VIGIE navigable, responsive, à la marque et internationalisé.

À l'issue de cette partie

- 1 Router VIGIE en SPA, avec deep linking vers un site/équipement.
- 2 Rendre l'interface responsive (toolkits, responsiveConfig).
- 3 Dériver un thème de marque via Fashion.
- 4 Externaliser les libellés et charger la locale.

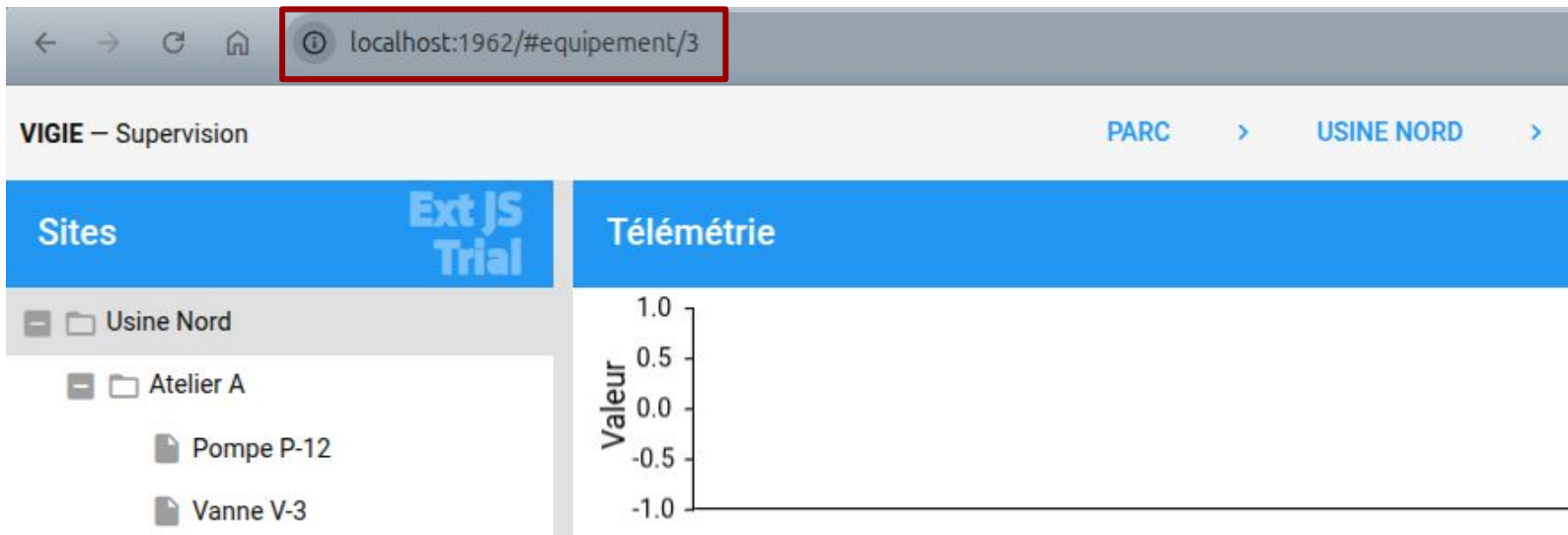
● BLOC 7.1

Routes & SPA

L'URL comme état de l'application

- **hashchange** ExtJS écoute le fragment d'URL (#...) et déclenche les routes.
- **routes** Un contrôleur associe un motif d'URL à une méthode.
- **Deep linking** Une URL partagée rouvre l'application sur le même état.
- **Bouton retour** La navigation s'intègre à l'historique du navigateur.

L'URL comme état de l'application



Déclarer des routes

```
Ext.define('VIGIE.controller.Navigation', {  
    extend: 'Ext.app.Controller',  
  
    routes: {  
        'equipment/:id': 'onEquipement',  
        'site/:nom':      'onSite'  
    },  
    onEquipement: function (id) { /* ... */ }  
});
```

Naviguer & contraindre

redirectTo met à jour l'URL par programme ; les conditions valident les paramètres de route.

```
// déclencher une route
this.redirectTo('equipement/2');

// contrainte sur le paramètre
routes: { 'equipement/:id': {
  action: 'onEquipement',
  conditions: { ':id': '[0-9]+' } } }
```

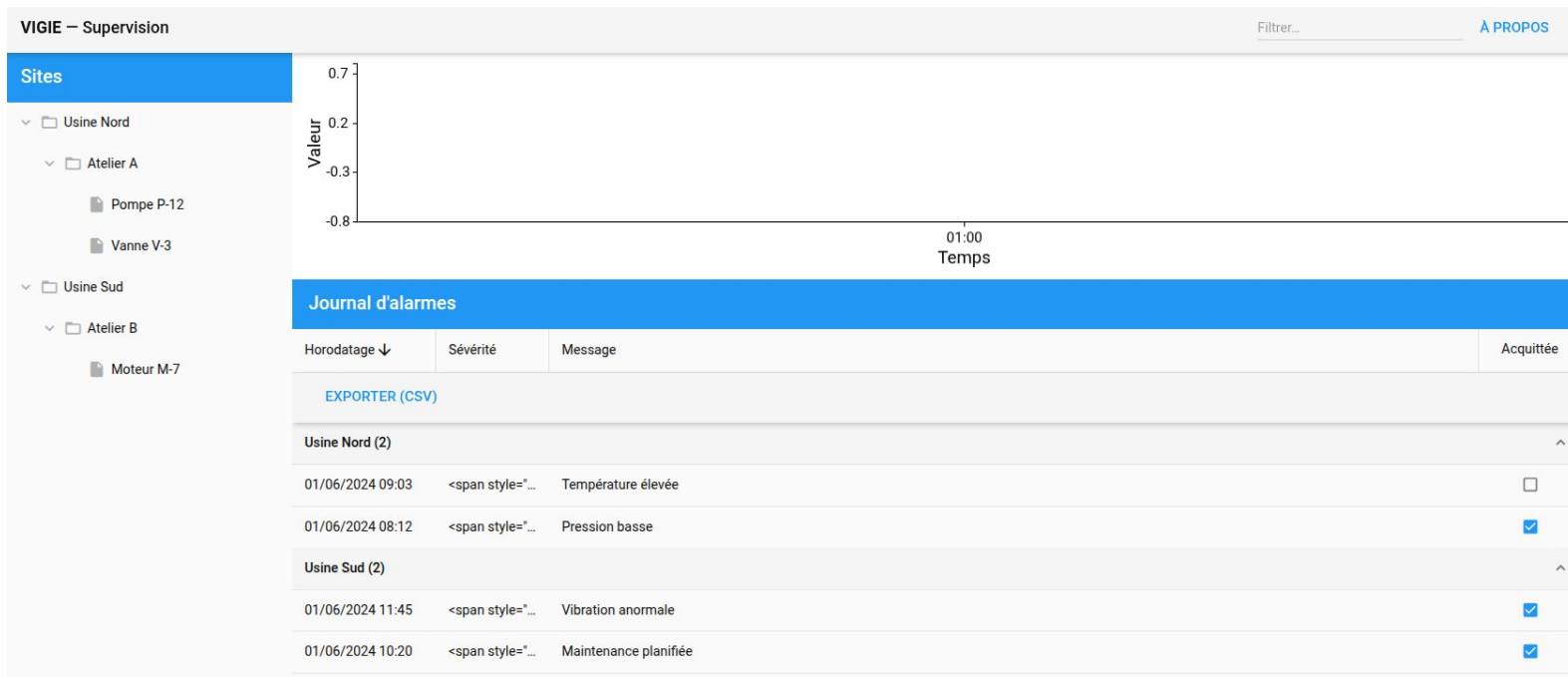
● BLOC 7.2

Responsive

Classic & modern : deux toolkits

- **classic** Riche, pensé desktop ; le choix de VIGIE (back-office).
- **modern** Tactile, mobile, transitions natives.
- **Code partagé** Le commun va dans app/ ; les surcharges dans classic/ et modern/.
- **Cible de build** Un build par toolkit, ou une app universelle servant les deux.

Modern : Illustration



S'adapter à l'espace & à la plateforme

responsiveConfig applique des configs selon des règles (largeur, orientation) ; platformConfig selon le type d'appareil.

```
plugins: 'responsive',
responsiveConfig: {
  'width < 800': { region: 'north', height: 120 },
  'width >= 800': { region: 'west', width: 280 }
}
```

Responsive Column Layout

Les colonnes se redistribuent selon la largeur : `columnWidth` en fraction, recalculée aux points de rupture.

```
{ layout: 'responsivecolumn', items: [  
  { columnWidth: 0.5, minHeight: 200 },  
  { columnWidth: 0.5, minHeight: 200 }  
] }
```

Idéal pour un dashboard de cartes qui passe de quatre colonnes à une sur mobile.

● BLOC 7.3

Thèmes

Thèmes : du Sass compilé

- **Sass** Les thèmes ExtJS sont décrits en variables et mixins Sass.
- **Fashion** Compile le Sass en direct pendant app watch : une variable change → rendu mis à jour.
- **Thèmes de base** Triton (clair, moderne), Neptune, Crisp, Material (modern).
- **Thème dérivé** Un thème de marque étend un thème de base et surcharge ses variables.

Thèmes : Exemple

The image displays three overlapping web form mockups, each representing a different theme: 'default' (blue), 'black' (dark), and 'green' (green). Each form contains the following elements:

- Header:** 'Theming Example by Saki' and a 'Theme' dropdown menu.
- Message Box:** A dropdown menu with 'Options'.
- Contact Information:** Fields for Name (First and Last), Email Address, and Phone Number.
- Mailing Address:** Fields for Street Address, City, State, and Postal Code.
- Navigation:** A set of tabs labeled 'Tab 1', 'Tab 2', and 'Tab 3'.
- Content:** A text area containing the placeholder text: 'Maecenas sed diam eget risus varius blandit sit amet non magna. Curabitur blandit tempus porttitor.'
- Buttons:** 'Reset' and 'OK' buttons.

A red modal dialog is shown over the 'default' form, and a green modal dialog is shown over the 'green' form. The modal dialogs have a title bar, a close button, and a 'Reset' button.

Dériver un thème de marque

Pointer app.json vers un thème dérivé, puis surcharger ses variables Sass - sans toucher au CSS compilé.

```
// app.json
"theme": "theme-vigie" // étend theme-triton

// theme-vigie/sass/var/all.scss
$base-color: #2BB3AD; // accent VIGIE
$base-highlight-color: #F2A340; // balise ambre
```

Jusqu'où personnaliser un thème ?

Sûr & durable

- Variables : couleurs, espacements, typographie.
- Mixins de composant pour des ajustements ciblés.
- Compatible avec les mises à jour du framework.

Fragile

- CSS brut plaqué par-dessus le thème.
- Sélecteurs visant la structure DOM interne.
- Casse silencieusement à la montée de version.

Personnaliser par les variables d'abord ; ne descendre au CSS qu'en dernier recours.

● BLOC 7.4

Internationalisation

Locale & libellés externalisés

Les packages de locale traduisent les composants natifs (dates, validations). Les libellés applicatifs, eux, ne doivent pas être codés en dur dans les vues.

```
// app.json
"locale": "fr"

// libellés centralisés (un singleton par langue)
Ext.define('VIGIE.Libelles', {
  singleton: true, JOURNAL: 'Journal d\'alarmes' });
```

● LAB M7

SPA routée & thématée

SPA routée, responsive & thémée

SOCLE - POUR TOUS

- Router VIGIE : deep-link vers un équipement.
- Rendre la mise en page responsive.
- Externaliser les libellés FR.
- Synchroniser l'URL avec la sélection.

EXTENSION - POUR ALLER PLUS LOIN

- Thème dérivé via Fashion (accent de marque).
- Second jeu de locale (EN) commutable.
- Adapter le dashboard au mobile.

État attendu en fin de lab

- VIGIE est navigable par URL : un lien ouvre le bon équipement.
- La mise en page s'adapte à la largeur de la fenêtre.
- Les libellés sont externalisés et la locale est chargée.

Resynchro : dépôt VIGIE de référence (état fin M7) fourni.

Ce qui est acquis, ce qui vient

Acquis - M7

- Routes & deep linking.
- Responsive & toolkits.
- Thème de marque (Fashion).
- Locale & libellés.

À venir - M8

- Build de production.
- Packages & app.json.
- Perf transverse.
- Tests (VM & stores).

● FIN DU Module 7

Place au Lab M7

Prochaine étape : Industrialisation (Module 8).



VIGIE

Industrialisation

ExtJS 8 · Module 8

Au programme

- **Bloc 8.1 - Build de production** Packages, app.json, microloader.
- **Bloc 8.2 - Tests** ViewModels, stores, proxies mockés.
- **Bloc 8.3 - Performance transverse** Bilan & check-list de production.
- **Bloc 8.4 - Pour aller plus loin** Optionnel : ExtReact / ExtAngular.

De « ça marche en dev » à « c'est livrable »

VIGIE est complète et fonctionne sous app watch. Reste à en faire un livrable de production fiable, testé et performant.



Objectif : build reproductible, tests sur la logique, et bilan de performance.

À l'issue de cette partie

- 1 Produire un build de production reproductible, packages recensés.
- 2 Tester la logique : formulas de ViewModel et stores (proxies mockés).
- 3 Établir une check-list de performance de production.
- 4 Situer ExtReact / ExtAngular (optionnel).

● BLOC 8.1

Build de production

Recenser les dépendances

- **Dette accumulée** Le M5 a ajouté exporter, le M6 charts, le M7 un thème et la locale.
- **app.json** Le manifeste déclare packages, thème, locale - la source de vérité du build.
- **Un package non requis** ...ne sera pas dans le bundle : la fonctionnalité manquera en prod.
- **requires propres** Le système de requires détermine ce qui entre - pas de code mort.

app.json & commandes de build

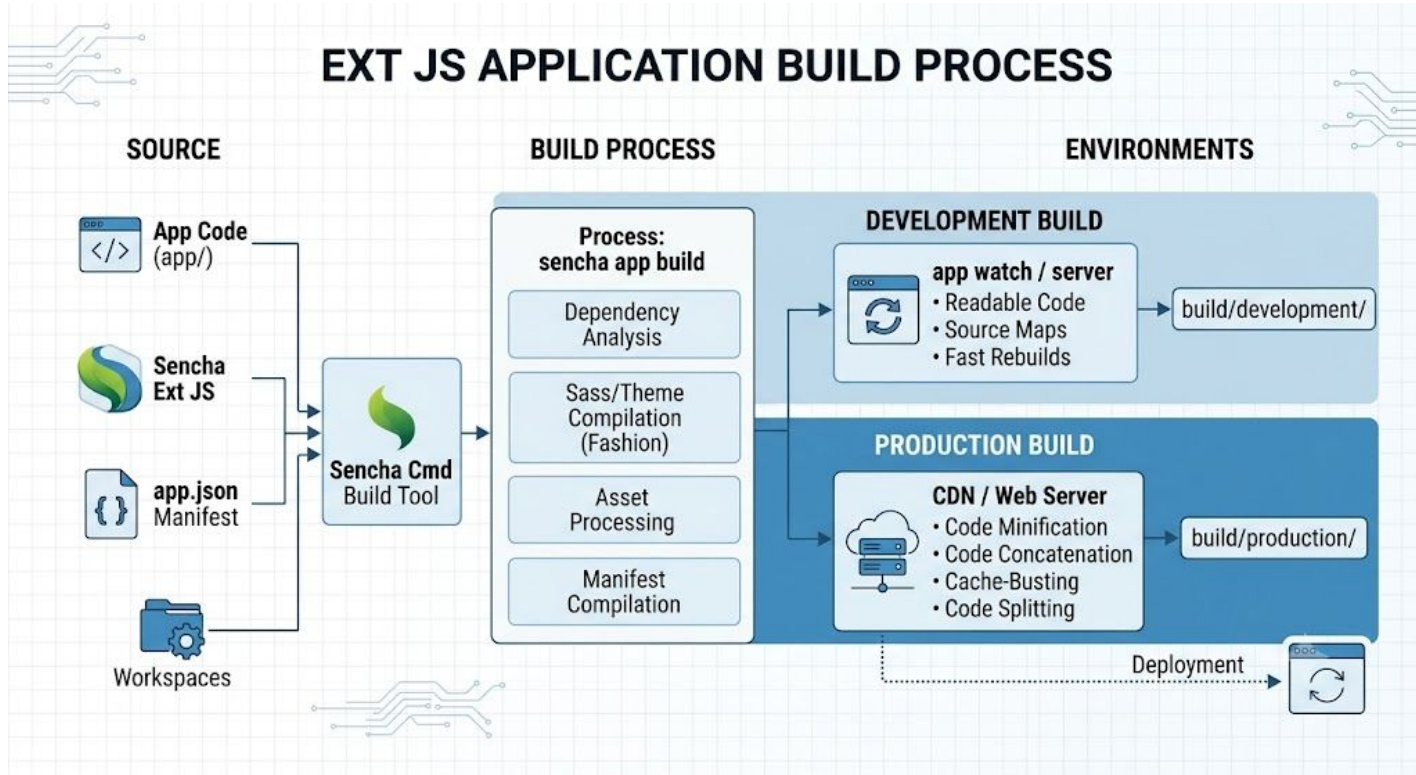
```
// app.json
"requires": [ "exporter", "charts" ],
"theme":     "theme-vigie",
"locale":    "fr"

$ sencha app build                # production
$ sencha app build development
```

Production, development, microloader

- **production** Minifié, concaténé, cache-busting : optimisé pour le réseau.
- **development** Lisible, source maps : pour diagnostiquer.
- **microloader** Amorce l'app à partir du manifeste compilé (app.json → bootstrap).
- **Code mort** Ce qui n'est pas atteint par requires n'entre pas dans le bundle.

Production, development, microloader



Livrable & environnements

- **build/production/VIGIE/** index.html, app.js, resources - servi par un serveur statique ou CDN.
- **URL d'API** Variabiliser l'endpoint : mock json-server en dev, API réelle en prod.
- **Profils** app.json gère des profils d'environnement (paramètres par cible).
- **Reproductible** Le même build, depuis les mêmes sources, produit le même livrable.

● BLOC 8.2

Tests

Quoi tester en priorité

- **La logique** Formulas, champs calculés de Model, transformations de store.
- **Les stores** Chargement, filtrage, tri, synchronisation.
- **Les ViewControllers** Réactions aux événements, sur des vues montées sans affichage.
- **Pas le pixel** Tester le comportement, pas le rendu exact - fragile et peu utile.

Tester une formula de ViewModel

notify() force l'évaluation synchrone des bindings : la formula est calculée et vérifiable.

```
it('enDefaut vrai si etat DEFAULT', function () {  
    var vm = new VIGIE.view.main.MainModel();  
    vm.set('equipementCourant',  
        new VIGIE.model.Equipement({ etat: 'DEFAULT' }));  
    vm.notify();  
    expect(vm.get('enDefaut')).toBe(true);  
});
```

Tester un store, proxy mocké

Un proxy memory isole le test du réseau : les données sont fournies en dur, le chargement reste synchrone. Sencha fournit des outils de test (Sencha Test), mais ces tests unitaires peuvent aussi être lancés avec des test runners standards comme Jasmine ou Mocha.

```
it('charge depuis le proxy', function () {  
    var store = Ext.create('VIGIE.store.Equipements', {  
        autoLoad: false,  
        proxy: { type: 'memory', data: [  
            { id: 1, nom: 'P-12' } ] }  
    });  
    store.load();  
    expect(store.getCount()).toBe(1);  
});
```

Sencha Test dans le pipeline

- **Unitaire** Jasmine pour la logique : rapide, exécuté à chaque commit.
- **End-to-end** WebDriver sur navigateurs réels pour les parcours critiques.
- **Parcours VIGIE** Sélectionner un équipement → journal filtré → acquitter.
- **CI** Intégrable à la chaîne d'intégration continue, en barrière de merge.

● BLOC 8.3

Performance transverse

Check-list de production

- **Build de prod** Minifié, requires propres, pas de code mort.
- **Grosses grilles** Buffered rendering et store paginé activés.
- **Renderers** Légers ; CSS plutôt que style inline calculé.
- **Démarrage** Mesurer le temps de boot ; charger en différé le non-critique.

Optimiser : quoi, quand

D'abord

- Mesurer : démarrage, scroll, mémoire.
- Profiler pour isoler la cause dominante.
- Définir un seuil acceptable.

Ensuite seulement

- Optimiser la cause prouvée.
- Re-mesurer pour valider le gain.
- S'arrêter quand le seuil est atteint.

Sur des mesures et des cas réels, pas sur du dogme - y compris pour la mise en production.

● BLOC 8.4 · OPTIONNEL

Pour aller plus loin

ExtReact & ExtAngular

Pertinent quand

- Une équipe déjà investie React / Angular.
- Besoin des grilles et charts Ext dans cet écosystème.
- Intégration ciblée, pas refonte.

À éviter quand

- Application ExtJS native (couche en trop).
- Attente d'un « port » gratuit : ça n'existe pas.
- Modèle mental et build différent.

Un teaser, pas une recommandation : situer l'outil, sans le survendre.

● LAB M8

Build & tests

Build & tests

SOCLE - POUR TOUS

- Recenser les packages dans app.json.
- Produire un build de prod qui démarre.
- Tester une formula du ViewModel.
- Tester un store (proxy mocké).

EXTENSION - POUR ALLER PLUS LOIN

- Tester un scénario maître-détail de bout en bout.
- Isoler un test par un mock de proxy dédié.
- Mesurer le temps de démarrage du build.

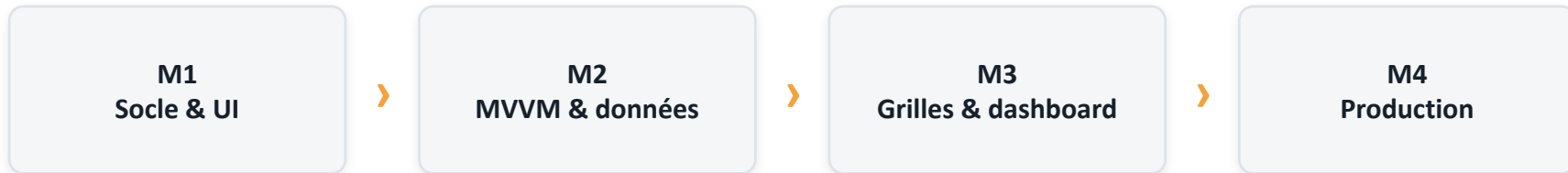
CHECKPOINT

État attendu en fin de formation

- Un build de production de VIGIE démarre, packages inclus.
- Un test de formula et un test de store passent.
- La check-list de performance a été parcourue.

Dépôt VIGIE final fourni : sources, mock, build et tests.

Le parcours VIGIE, de bout en bout



D'une classe vide à une console de supervision routée, testée et buildée

- Système de classes, layouts, MVVM, couche données.
- Grilles performantes, arbres, charts, dashboard piloté par la sélection.
- SPA routée, responsive, thémée, internationalisée, buildée et testée.

● FIN DE LA FORMATION

VIGIE est en production

Merci - place aux questions et à la revue d'architecture.